

THE UNIVERSITY OF DANANG
DANANG UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

GRADUATION PROJECT THESIS

MAJOR: INFORMATION TECHNOLOGY

SPECIALTY: INFORMATION SYSTEM

PROJECT TITLE:

**SQLTUNER - AI ASSISTANT
FOR AUTOMATING DATABASE
PERFORMANCE OPTIMIZATION**

Instructor: **Dr. NINH KHÁNH DUY**

Student: **NGUYỄN LÊ QUỐC HUY**

Student ID: **102220191**

Class: **22T_DT4**

Da Nang, 6/2026

**THE UNIVERSITY OF DANANG
DANANG UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY**

GRADUATION PROJECT THESIS

**MAJOR: INFORMATION TECHNOLOGY
SPECIALTY: INFORMATION SYSTEM**

PROJECT TITLE:

**SQLTUNER - AI ASSISTANT
FOR AUTOMATING DATABASE
PERFORMANCE OPTIMIZATION**

Instructor: **Dr. NINH KHÁNH DUY**
Student: **NGUYỄN LÊ QUỐC HUY**
Student ID: **102220191**
Class: **22T_DT4**

Da Nang, 6/2026

SUPERVISOR'S COMMENTS

REVIEWER'S COMMENTS

SUMMARY

Topic title: SQLTuner - AI Assistant for Automating Database Performance Optimization

Student name: Nguyễn Lê Quốc Huy

Student ID: 102220191

Class: 22T_DT4

This graduation project presents the research and development of **SQLTuner**, an advanced AI-powered assistant system designed to automate the entire lifecycle of SQL queries, encompassing generation, optimization, and evaluation. As data-driven decision-making becomes ubiquitous, the technical barrier of mastering Structured Query Language (SQL) remains a significant challenge for non-technical personnel. To address this, SQLTuner leverages Local Large Language Models (LLMs) to accurately translate natural language intentions into executable SQL commands. By prioritizing local deployment, the system ensures strict data privacy and security, completely eliminating the risks associated with exposing sensitive database schemas to third-party cloud APIs.

Beyond standard Text-to-SQL translation, the system features a deep integration with database performance tuning mechanics. SQLTuner automatically analyzes query execution plans to identify computational bottlenecks and proposes actionable refactoring strategies to enhance performance. To guarantee the safe execution of AI-generated code, the project introduces a virtual sandbox environment (Simulation Mode). This zero-mutation workspace allows users to prototype database schemas and securely test complex queries without risking real production data.

Furthermore, a comprehensive automated evaluation framework is implemented to systematically quantify the AI model's proficiency. Utilizing standardized, high-complexity academic benchmarks such as the Spider and BIRD datasets, the system measures reliability based on rigorous Execution Accuracy (EX) and Exact Match (EM) metrics. Extensive testing results demonstrate the system's high stability, secure execution environment, and advanced capability to optimize resource-intensive queries. Ultimately, SQLTuner proves to be a highly practical, privacy-preserving toolset that streamlines database management workflows and significantly lowers the barrier to data interaction.

GRADUATION PROJECT REQUIREMENTS

Student Name: Nguyễn Lê Quốc Huy

Student ID : 102220191

Class: 22T_DT4 Faculty: Information Technology Major: Information Technology

1. *Topic title:*

SQLTuner - AI Assistant for Automating Database Performance Optimization

Project topic : ☐ has signed intellectual property agreement for final result

2. *Initial figure and data:*

Standardized academic benchmark datasets: Spider and BIRD (A Big Bench for Large-Scale Database Grounded Text-to-SQLs).

Database performance inputs: PostgreSQL Execution Plans generated via EXPLAIN and EXPLAIN ANALYZE commands.

AI parameters: Local Large Language Models (such as Qwen-2.5-Coder and Llama-3) configurations including temperature and token limits.

Content of the explanations and calculations:

Chapter 1: Theoretical foundation of the Text-to-SQL problem, the application of LLMs in database code generation, and mathematical evaluation metrics including Execution Accuracy (EX) and Exact Match (EM).

Chapter 2: System analysis including Use Case definitions, overall system architecture, relational database design, and the construction of the safe Database Simulation (Sandbox) mechanism and Evaluation Pipeline workflow.

Chapter 3: Implementation details involving the Docker deployment model, UI/Admin interface configurations, and the evaluation of experimental results across five distinct performance scenarios.

3. *Drawings, charts (specify the types and sizes of drawings):*

Use Case Diagram

Activity Diagram

Sequence Diagram

4. *Name of instructor: Dr. Ninh Khánh Duy*

5. *Date of assignment :/03/2026*

6. *Date of completion :/06/2026*

Đà Nẵng, date month year 2026

Head of Division.....

Instructor

PREFACE

In the era of digital transformation and big data management, extracting information accurately and efficiently from Relational Database Management Systems is a significant challenge for software and data engineers. The project "SQLTuner - AI Assistant for Automating Database Performance Optimization" was conducted with the ambition to harness the power of Large Language Models (LLMs) to solve this problem comprehensively.

To complete this graduation project, I would like to express my deepest gratitude to the lecturers of the Faculty of Information Technology at Da Nang University of Science and Technology for their dedicated teaching and for providing a solid foundation of knowledge. Special thanks to my thesis instructor for the continuous guidance, orientation, and valuable advice throughout the research and development process. I also want to extend my sincere appreciation to the Data Team at DAC Data Technology Vietnam for their practical support, sharing of industry experience, and creating an environment that fostered my problem-solving skills during this period.

Despite my best efforts, limitations in time and experience mean this project may still have shortcomings. I warmly welcome any feedback and suggestions from the evaluators to further improve this research.

ASSURANCE

I, Nguyễn Lê Quốc Huy, student of class 22T_DT4, hereby declare that the graduation project titled "SQLTuner - AI Assistant for Automating Database Performance Optimization" is my own original research, conducted under the supervision of my instructor. The data, architectures, and evaluation results presented in this report are entirely truthful and have not been published in any other work.

All references and citations from documents, articles, source codes, or benchmark datasets are clearly and accurately acknowledged in the References section. I take full responsibility before the university and the law for the academic integrity of this work.

Student Performed

Nguyễn Lê Quốc Huy

TABLE OF CONTENTS

SUPERVISOR'S COMMENTS	3
REVIEWER'S COMMENTS	4
SUMMARY.....	5
PREFACE.....	i
ASSURANCE	ii
TABLE OF CONTENTS	iii
LIST OF TABLES AND PICTURES	v
LIST OF SYMBOLS AND ACRONYMS	vii
INTRODUCTION.....	4
CHAPTER 1: THEORETICAL BASIS	5
1.1. Overview of the Text-to-SQL Problem	5
1.1.1. Concept and Role	5
1.1.2. Core Challenges	5
1.2. Application of Large Language Models (LLMs) in Databases	5
1.2.1. Code Generation Capabilities	5
1.2.2. Optimization via Prompt Engineering	5
1.2.3. Query Optimization.....	6
1.3. Evaluation Benchmarks	6
1.3.1. Standard Evaluation Benchmarks	6
1.3.2. Core Evaluation Metrics	7
1.4. Conclusion of Chapter 1	7
CHAPTER 2: ANALYSIS AND DESIGN.....	8
2.1. Problem Statement.....	8
2.2. Current Situation Analysis.....	8
2.3. Functional Analysis	8
2.3.1. Actors	9
2.3.2. Generation and SQL Optimization Subsystem	9
2.3.3. Evaluation Framework Module (Core Component)	9
2.3.4. Database Simulation / Sandbox Subsystem	10
2.3.5. Technologies Used and Selection Rationale.....	10

2.4.	System Diagram.....	10
2.4.1.	Use Case Diagram.....	10
2.4.2.	Activity Diagram.....	21
2.4.3.	Sequence Diagram	25
2.5.	Database Design	30
2.5.1.	Overall Diagram.....	30
2.5.2.	Details of Core Data Tables	31
2.6.	System Construction	34
2.6.1.	Project Folder Structure Organization	34
2.6.2.	Database Simulation (Sandbox Environment) Mechanism Design.....	34
2.6.3.	Evaluation Pipeline Workflow.....	34
2.7.	Conclusion of Chapter 2	35
CHAPTER 3: DEPLOYMENT AND EVALUATION RESULTS		36
3.1.	Deployment Model	36
3.1.1.	Deployment Environment.....	36
3.1.2.	Tools Used	36
3.1.3.	System Configuration	36
3.2.	System User Interface.....	37
3.2.1.	User Interface	37
3.2.2.	Admin Interface	52
3.3.	Experimental Results	59
3.3.1.	Scenario 1: Basic Text-to-SQL Query Generation	59
3.3.2.	Scenario 2: Complex Query Optimization.....	59
3.3.3.	Scenario 3: Batch Inference on Benchmarks	59
3.3.4.	Scenario 4: Extracting Evaluation Reports	59
3.3.5.	Scenario 5: Safe Execution with Database Simulation / Sandbox.....	59
3.4.	Result Evaluation Remarks.....	59
CONCLUSION		61
REFERENCES		63

LIST OF TABLES AND PICTURES

Table 2.1: Overview of Use Case Groups	12
Table 2.2: Account and Session Management Use Case Specification	14
Table 2.3: Database Connection Management Use Case Specification.....	16
Table 2.4: Conversation and SQL Query Use Case Specification	18
Table 2.5: Performance Analysis and Optimization Use Case Specification	20
Table 2.6: db_connections Table Structure Specification.....	31
Table 2.7: query_logs Table Structure Specification	32
Table 2.8: performance_analysis Table Structure Specification.....	33
Picture 2.1: Overall Use Case Diagram of the SQLTuner System	11
Picture 2.2: Account and Session Management Use Case Diagram	13
Picture 2.3: Database Connection Management Use Case Diagram.....	15
Picture 2.4: Conversation and SQL Query Use Case Diagram	17
Picture 2.5: Performance Analysis and Optimization Use Case Diagram	19
Picture 2.6: Login and Authentication Activity Diagram	21
Picture 2.7: Add Connection and Schema Synchronization Activity Diagram	22
Picture 2.8: Automated SQL Generation (Text-to-SQL) Activity Diagram.....	23
Picture 2.9: Query Analysis and Optimization Activity Diagram.....	24
Picture 2.10: SQL Execution on Sandbox Sequence Diagram	25
Picture 2.11: Text-to-SQL Parsing and Auto-generation Sequence Diagram	26
Picture 2.12: Explain Query Plan Sequence Diagram.....	27
Picture 2.13: SQL Optimization via LLM Sequence Diagram	28
Picture 2.14: Local Schema Editing and Synchronization Sequence Diagram.....	29
Picture 2.15: Database Schema Diagram	30
Picture 3.1: User Login and Authentication Interface.....	37
Picture 3.2: Main Overview Dashboard and Activity Tracking.....	38
Picture 3.3: Workspaces and Database Connection Management Dashboard.....	39
Picture 3.4: Create New Workspace Configuration Modal.....	40
Picture 3.5: Edit Workspace and Synchronization Modal	40
Picture 3.6: Intelligent Text-to-SQL Chat Interface Layout	41
Picture 3.7: SQL Generation, Execution Results, and Schema Inspector.....	41
Picture 3.8: Interactive Database Schema Diagram Visualization.....	42

Picture 3.9: User-AI Chat Interaction for SQL Generation.....	43
Picture 3.10: Edit Generated SQL Modal for Manual Refinement.....	44
Picture 3.11: SQL Query Execution and Result Set Rendering	45
Picture 3.12: Query Execution Plan Analysis Modal (EXPLAIN)	46
Picture 3.13: Query Execution Plan Analysis Modal (EXPLAIN ANALYZE)	46
Picture 3.14: AI-Driven Optimization Analysis and Refactoring Proposal	47
Picture 3.15: Creating a Virtual Simulation Workspace	48
Picture 3.16: Manual Database Schema Editor for Simulation.....	48
Picture 3.17: Sample Data Generation and Management for Simulation	49
Picture 3.18: Interactive Visual Diagram Editor for Schema Design	50
Picture 3.19: Global History and Activity Audit Log	51
Picture 3.20: Admin Portal System Overview Dashboard	52
Picture 3.21: Admin Evaluation Dashboard and Benchmark Performance Metrics.....	53
Picture 3.22: Success Evaluation Results and SQL Comparison on Spider Benchmark	54
Picture 3.23: Failed Evaluation Cases and Error Analysis on Spider Benchmark.....	54
Picture 3.24: User Management and Role Assignment Control Panel	55
Picture 3.25: Global Database Connections Management Panel	56
Picture 3.26: Evaluation Runner for Per-Database Benchmark Execution.....	57
Picture 3.27: Detailed Evaluation Results Modal for Specific Databases	57
Picture 3.28: Proposed AI Training and User Feedback Review Dashboard (RLHF) ..	58

LIST OF SYMBOLS AND ACRONYMS

SYMBOL:

- **R_{pred}** : Predicted Result Set – The dataset returned after executing the AI-generated SQL query.
- **R_{true}** : Standard Result Set (Ground Truth) – The accurate dataset returned after executing the correct standard SQL query.
- **EX**: Execution Accuracy – The evaluation metric measuring the functional correctness of the generated SQL.
- **EM**: Exact Match – The evaluation metric measuring the strict syntactic string match between the generated and target SQL.

ACRONYM:

- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **DBA**: Database Administrator
- **DDL**: Data Definition Language
- **DML**: Data Manipulation Language
- **IDE**: Integrated Development Environment
- **JSON / JSONB**: JavaScript Object Notation / JSON Binary
- **LLM**: Large Language Model
- **NLP**: Natural Language Processing
- **OOM**: Out of Memory
- **RDBMS**: Relational Database Management System
- **RLHF**: Reinforcement Learning from Human Feedback
- **SPA**: Single Page Application
- **SQL**: Structured Query Language
- **UI**: User Interface

INTRODUCTION

In the era of digitalization and big data management, Structured Query Language (SQL) plays a crucial role in extracting information from Relational Database Management Systems (RDBMS), particularly PostgreSQL. However, building, testing, and optimizing complex SQL commands often requires significant time and deep technical expertise from Data Engineers and Backend Developers.

The rise of Large Language Models (LLMs) has opened a new direction for automating SQL generation (Text-to-SQL). Nevertheless, current solutions still lack performance optimization, automated evaluation frameworks, and regression control.

Therefore, this project aims to build SQLTuner – a comprehensive platform that integrates three core processes: Intelligent Query Generation, Performance Optimization based on actual execution plans, and Automated Batch Evaluation to manage the model's quality lifecycle.

CHAPTER 1: THEORETICAL BASIS

1.1. Overview of the Text-to-SQL Problem

1.1.1. Concept and Role

Text-to-SQL (converting text into SQL Queries) is an important problem in the field of Natural Language Processing (NLP) combined with Databases. The goal of the problem is to automate the translation of Natural Language questions and requests from users into valid and executable Structured Query Language (SQL) statements on Relational Database Management Systems (RDBMS).

This process acts as a bridge, helping to eliminate technical barriers. Instead of having to understand SQL syntax, Database Schema, and complex data manipulation functions, end users (business staff, managers) simply need to ask questions in ordinary language (e.g., "Show me the top 5 employees with the highest sales in Q1 2026").

1.1.2. Core Challenges

Despite providing high practical value, Text-to-SQL is a challenging problem due to the significant difference between the flexibility of Natural Language and the strict rules of programming languages. The main challenges include:

- **Ambiguity:** A question can be expressed in many different ways or lack clear keywords to map directly to table/column names in the Database.
- **Schema Linking:** The system's ability to correctly identify the entities mentioned in the question and map them accurately to the corresponding tables, columns, and foreign keys in the Database. This is a decisive step for the success of the query.
- **Complex Query Structures:** For large Databases, extracting information often requires nested operations (Nested Queries), multiple table connections (Multiple JOINS), data grouping (GROUP BY), and complex condition filtering (HAVING, nested conditional clauses).
- **Cross-domain Generalization:** A good Text-to-SQL system must be able to operate on new Databases (zero-shot) that it has never been trained on before.

1.2. Application of Large Language Models (LLMs) in Databases

1.2.1. Code Generation Capabilities

Modern LLMs are trained in a massive amount of data including both source code and text. As a result, they possess outstanding logical reasoning and Code Generation capabilities. For SQL, instead of having to build manual parsers like traditional methods, LLMs can learn the relationship between natural questions and corresponding SQL syntax.

1.2.2. Optimization via Prompt Engineering

For LLMs to generate accurate SQL statements for a specific Database, Prompt Engineering techniques are heavily applied. Through the Prompt design process, the system provides the LLM with:

- **Metadata:** Includes Data Definition Language (DDL), table names, column names, data types, and primary key/foreign key descriptions.
- **Few-shot Prompting:** Providing a few example pairs (Natural Language question - SQL statement) to guide the model to learn the desired format.
- **Chain-of-Thought Reasoning:** Requiring the model to explain step-by-step logic before writing SQL, helping to enhance its ability to handle complex requests.

1.2.3. Query Optimization

Besides generating queries, LLMs are also applied to optimize Database performance. By providing information from the Execution Plan obtained through commands like EXPLAIN or EXPLAIN ANALYZE, LLMs can analyze bottlenecks—such as costly Sequential Scans. From there, the AI can propose optimization options such as rewriting the JOIN clause structure, adding appropriate Indexes, or changing the data filtering method to reduce resource costs and response time.

1.3. Evaluation Benchmarks

To ensure objectivity and accurately measure the performance of Text-to-SQL systems and strategies, the use of Evaluation Benchmarks is mandatory. The project focuses on researching and using two of the leading academic datasets today.

1.3.1. Standard Evaluation Benchmarks

Spider Benchmark:

- **Characteristics:** It is one of the most popular and large-scale Text-to-SQL Evaluation Benchmarks. Spider stands out with its Cross-domain characteristic, containing thousands of Databases belonging to many different fields.
- **Challenges:** It poses a strict zero-shot testing environment (the model does not see the test Database during training) and categorizes query difficulty into 4 levels: Easy, Medium, Hard, and Extra Hard. Hard-level statements often require multiple JOINS, nested subqueries, and set operations.

BIRD Benchmark (A Big Bench for Large-Scale Database Grounded Text-to-SQLs):

- **Characteristics:** This is a next-generation dataset, designed to bridge the gap between the academic environment and industrial practice.
- **Challenges:** BIRD focuses on very large-scale Databases, messy data, and requires the system to have a deep understanding of internal data semantics (database-grounded). Moreover, to solve the questions in BIRD, the model often needs to combine with an external knowledge base—accurately simulating how a Data Engineer works in reality.

1.3.2. Core Evaluation Metrics

Model results cannot merely be evaluated by the naked eye but need to be measured using strict mathematical formulas. The system uses two main metrics:

Execution Accuracy (EX):

- **Definition:** This is the gold standard in Text-to-SQL evaluation. The system will execute both the AI-generated query (Predicted SQL) and the correct answer query (Ground Truth SQL) on the same real Database state.
- **Evaluation:** If the Result Set returned from these two statements perfectly matches, the query is evaluated as correct (Pass). This criterion is very fair and realistic because a business requirement can entirely be solved with various SQL syntaxes.

Exact Match (EM):

- **Definition:** Evaluates the degree of absolute match in form (syntax) between the Predicted SQL and the Ground Truth SQL.
- **Evaluation:** Before comparison, both statements undergo a normalization process (removing extra whitespaces, synchronizing uppercase/lowercase, rearranging the order of logically independent conditions). This index helps check the model's ability to imitate strict structures, but often has lower scores than EX as it does not account for the diversity in writing queries.

1.4. Conclusion of Chapter 1

Chapter 1 has provided a comprehensive overview of the theoretical foundation of the Text-to-SQL Problem. Ranging from defining and identifying Core Challenges, to analyzing the massive potential of Large Language Models (LLMs) in automating and optimizing Database Queries. At the same time, the chapter also established strict measurement standards via reputable Evaluation Benchmarks (Spider, BIRD) and metrics (EX, EM). This solid theoretical system serves as an important premise to justify the architectural, analytical, and design solutions of the SQLTuner system that will be detailed in Chapter 2.

CHAPTER 2: ANALYSIS AND DESIGN

2.1. Problem Statement

In the era of digitalization and big data management, Structured Query Language (SQL) plays a paramount role in extracting information from Relational Database Management Systems (RDBMS), particularly PostgreSQL. However, building, testing, and optimizing complex SQL statements often requires Data Engineers and Backend developers to expend significant time and deep expertise in storage infrastructure.

The rise of Large Language Models (LLMs) opens a new direction for automating Code Generation for SQL (Text-to-SQL). Nevertheless, current solutions on the market remain fragmented and reveal three major bottlenecks:

- **Lack of performance optimization:** Most tools only stop at generating syntactically valid SQL (Semantics) but lack the ability to evaluate or refine the statement to achieve the best Execution Performance in a real environment.
- **Lack of automated testing mechanisms:** There is no standard integrated framework allowing for automated testing and Batch Evaluation of the quality of AI-generated statements based on standard Evaluation Benchmarks.
- **Difficulty in controlling Regression errors:** When developers change the Prompt Engineering structure or change the underlying LLM model, the system lacks quantitative measurement tools to know if the overall accuracy has increased or decreased.

The requirement is to build the SQLTuner system—a comprehensive platform that tightly integrates three core processes: Intelligent Query Generation, Performance Optimization based on actual Execution Plans, and Automated Batch Evaluation to manage the model's quality lifecycle.

2.2. Current Situation Analysis

A practical survey of current programming support tools (such as GitHub Copilot, ChatGPT, or AI extensions on IDEs) shows that these tools assist developers relatively well at the single-turn query creation level. However, for professional Data Engineering workflows, these solutions reveal obvious limitations:

- **Disconnection from Database states:** Ordinary AI assistants cannot perceive table sizes, the presence of Indexes, or actual data distribution in the Database. Thus, generated statements can easily fall into a Timeout or Sequential Scan situation when facing millions of records.
- **Unaddressed accuracy testing:** Engineers must manually copy the SQL statement from the AI chat window, paste it into a database client to run and verify the returned Result Set. This manual process cannot be applied when needing to test model accuracy on a scale of thousands of questions across different knowledge domains before pushing the system to Production.

2.3. Functional Analysis

2.3.1. Actors

The SQLTuner system targets two main user groups with corresponding goals:

- **Backend Developers / Database Administrators (DBA):** This group uses the system to quickly convert business requirements into SQL code, while leveraging the optimization module to analyze slow queries, thereby receiving performance improvement recommendations (such as restructuring JOINS or adding Indexes).
- **AI Researchers / Data Engineers:** This group uses the Evaluation Framework module as a specialized tool to quantitatively measure and score the performance of different LLMs or evaluate new Prompt Engineering techniques based on standard datasets.

2.3.2. Generation and SQL Optimization Subsystem

- **Text-to-SQL Generation:** The system extracts Database Schema information including table names, columns, data types, and primary/foreign key constraints. This information is automatically structured into a Context Prompt sent to the LLM alongside the user's Natural Language question to retrieve the corresponding SQL Query.
- **SQL Optimization:** The system receives the SQL Query needing optimization, implicitly executes the EXPLAIN (ANALYZE, FORMAT JSON) command on the RDBMS to retrieve the detailed Execution Plan tree. The Optimizer subsystem will extract metrics on Cost, Actual Time, and heavy memory operations. Based on this, the AI module will propose a specific new query to optimize resources.

2.3.3. Evaluation Framework Module (Core Component)

This module acts as an Automated Testing Framework running in the background or triggered via the interface:

- **Manage and Load Dataset Benchmarks:** Supports importing standardized structured datasets like Spider and BIRD, including a list of Natural Language questions, corresponding target Databases, and standard answer queries (Ground Truth SQL).
- **Batch Inference Execution:** Automatically allocates the entire list of questions in the dataset to process through locally deployed language models (Local LLMs) via asynchronous mechanisms. The system integrates Queue Management algorithms and controls Batch Size to optimize hardware performance (CPU/GPU) and prevent Out of Memory (OOM) scenarios during execution.
- **Evaluation Engine and Error Analysis:** Executes both SQL Queries (Predicted SQL and Ground Truth SQL) in parallel within an isolated, secure Sandbox environment, comparing the returned dataset results to accurately calculate two metric scores: Execution Accuracy (EX) and Exact Match (EM). Syntax Errors or Runtime Errors are captured in detail along with error messages from the Database system to serve Debugging purposes.

2.3.4. Database Simulation / Sandbox Subsystem

To ensure absolute safety when automatically executing AI-generated source code, the system requires a Database Simulation / Sandbox environment module. This module is responsible for:

- **Execution Isolation:** Creating temporary schemas or using Transaction Rollback mechanisms to test-run Predicted SQL statements. This completely prevents data loss risks when the LLM generates incorrect syntax containing destructive commands (like DROP, TRUNCATE, UPDATE without a WHERE clause).
- **Data Mocking:** Supports quickly loading a representative data subset from the original Database into the Sandbox environment, helping the Result Match operation occur accurately without the cost of scanning the entire actual large dataset.
- **Lifecycle Management:** Automatically tearing down and reclaiming memory resources immediately after finishing the result comparison for one question, returning to a clean original state for the next evaluation turn.

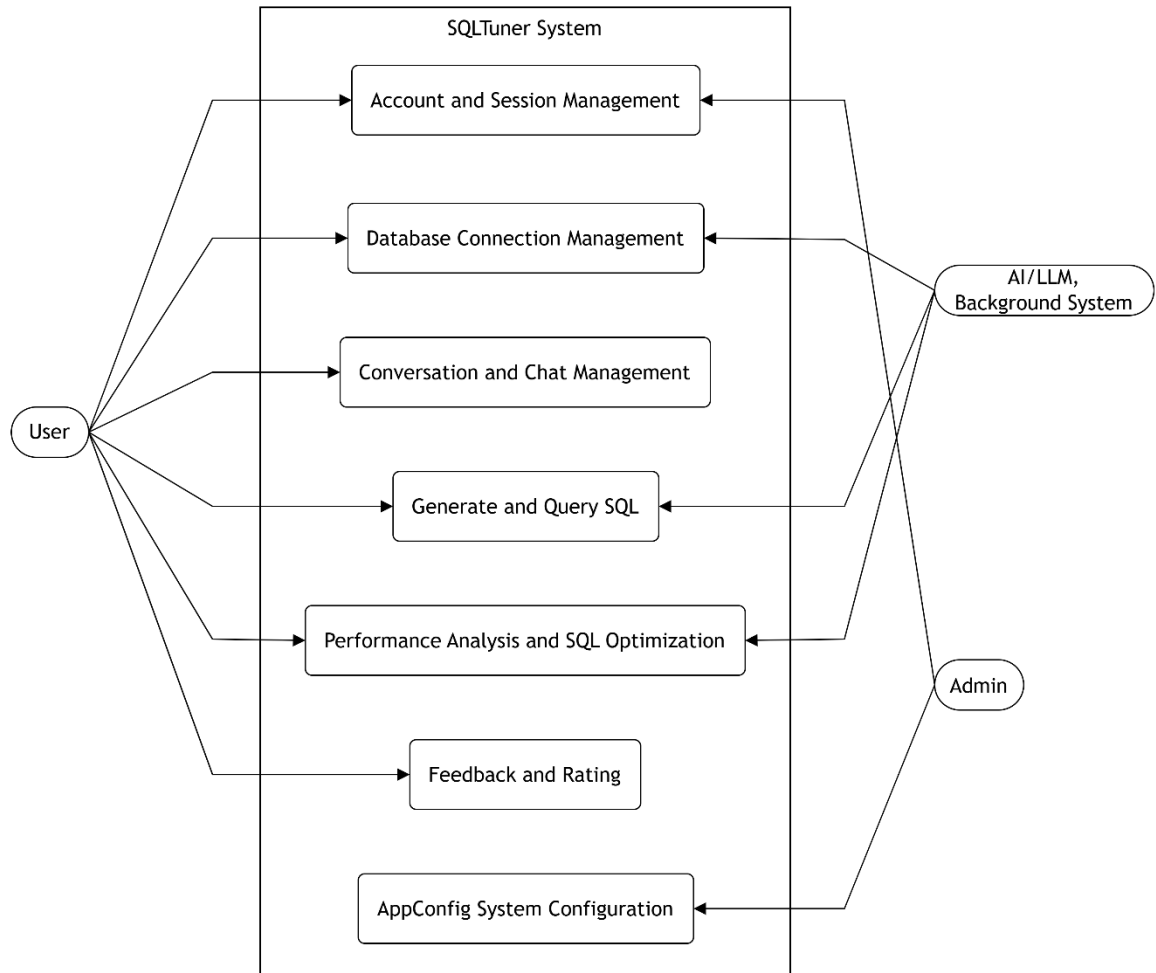
2.3.5. Technologies Used and Selection Rationale

The system chooses modern technologies to ensure clear source code separation and high operational performance:

- **Backend (Python & FastAPI):** Python is the standard language for AI and data processing tasks. FastAPI is chosen thanks to its ASGI mechanism supporting robust asynchronous programming (async/await), optimizing performance when the system has to perform thousands of concurrent I/O-bound requests (calling LLMs and querying Databases continuously during Evaluation).
- **Database (PostgreSQL):** A powerful, open-source Relational Database Management System supporting deep Execution Plan analysis features that perfectly serve the optimization module.
- **Frontend (React, Vite, TypeScript):** Building a smooth Single Page Application (SPA) interface. Vite accelerates build and hot-reload speeds during development; TypeScript ensures data type strictness, minimizing UI runtime errors when interacting with complex log structures.

2.4. System Diagram

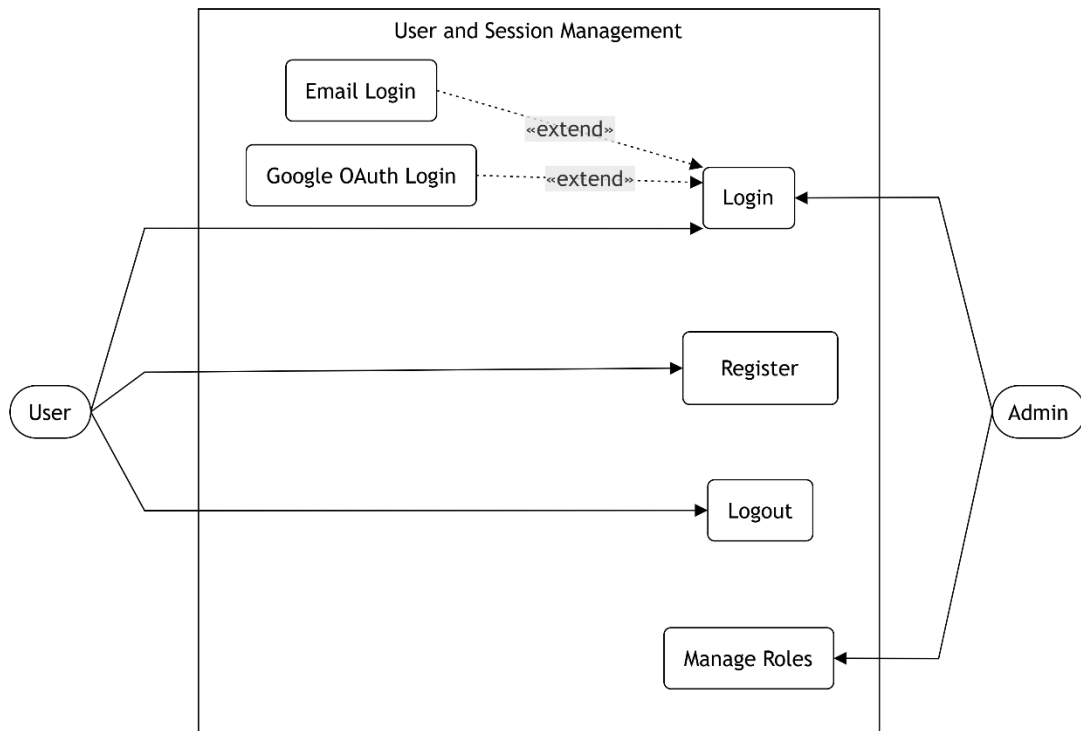
2.4.1. Use Case Diagram



Picture 2.1: Overall Use Case Diagram of the SQLTuner System

Table 2.1: Overview of Use Case Groups

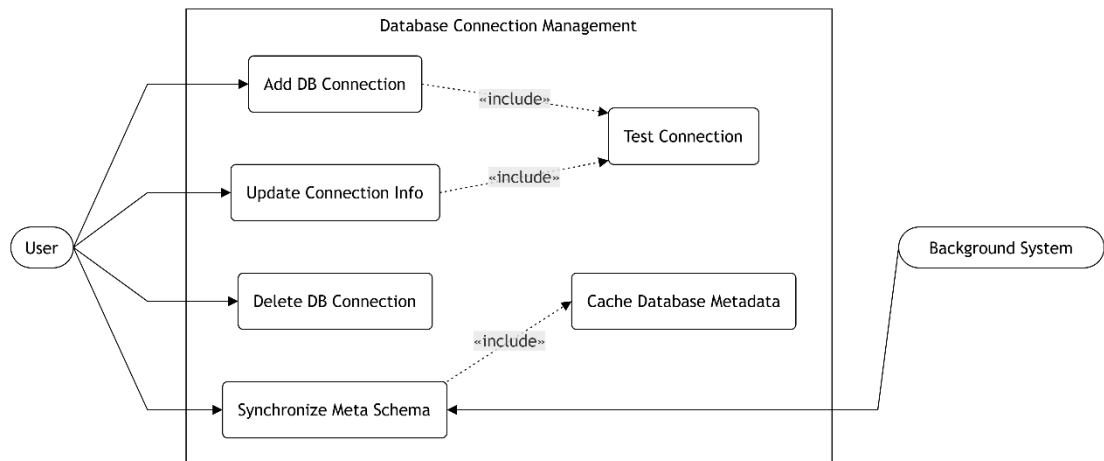
Use Case Group	Actor	Description	Main Goal
Account and Session Management	User, Admin	Handles user registration, authentication, roles, and session persistence.	Secure user access.
Database Connection Management	User, System	Manages database credentials, configurations, and metadata synchronization.	Maintain active connection links.
Conversation and Chat Management	User	Allows users to interact with the system via natural language prompts.	Facilitate context-aware chat.
Generate and Query SQL	User, System	Translates natural language to SQL and executes it on target databases.	Return accurate data results.
Performance Analysis and SQL Optimization	User, System	Analyzes explain plans and suggests LLM-based SQL optimizations.	Improve query performance.
Feedback and Rating	User	Collects user ratings and manually corrected SQL for model fine-tuning.	Gather reinforcement data.
AppConfig System Configuration	Admin	Global settings management for the application.	Customize system behavior.



Picture 2.2: Account and Session Management Use Case Diagram

Table 2.2: Account and Session Management Use Case Specification

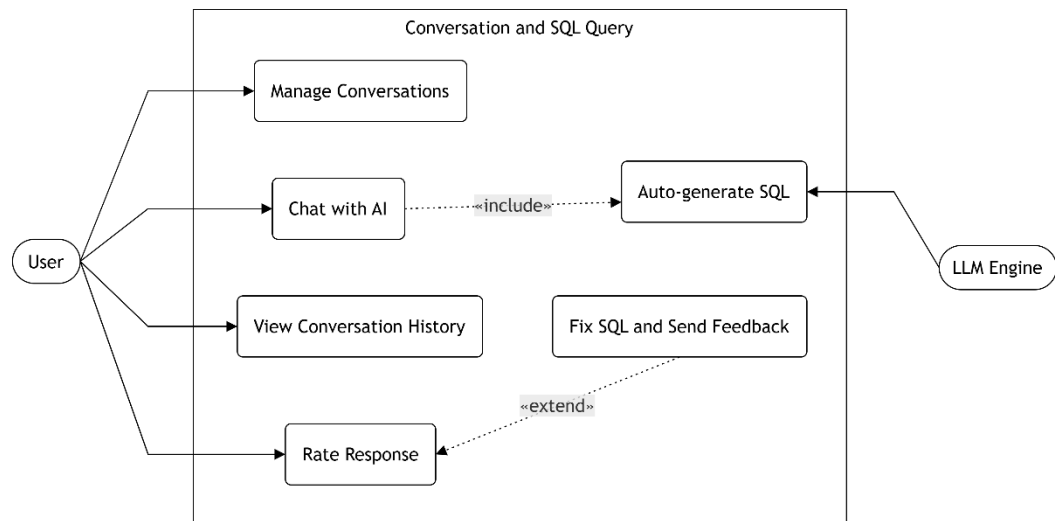
Use Case Name	Actor	Preconditions	Main Flow	Postconditions
Register Account	User	None	User inputs email and password; system validates and creates a new user record.	User account is created and ready for login.
Login	User	Account exists	User selects login method (Email or Google); system validates credentials and initializes session.	User is authenticated and redirected to dashboard.
Logout	User	User is logged in	User requests logout; system revokes the active session token.	User is logged out and redirected to login page.
View Profile	User	User is logged in	User accesses profile page; system retrieves and displays user info (avatar, email).	Profile info is displayed correctly.
Manage Roles	Admin	Admin is logged in	Admin selects a user to change role; system updates the UserRole enum in DB.	User's permissions are updated.



Picture 2.3: Database Connection Management Use Case Diagram

Table 2.3: Database Connection Management Use Case Specification

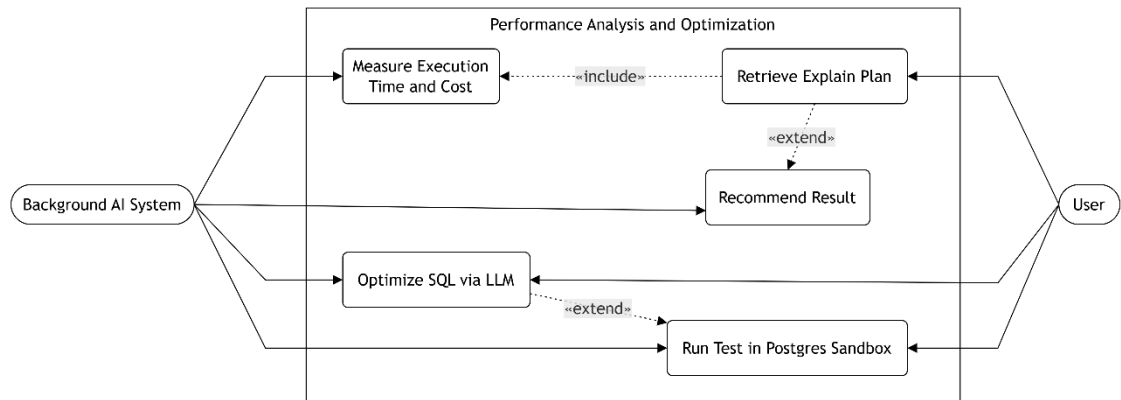
Use Case Name	Actor	Preconditions	Main Flow	Postconditions
Add DB Connection	User	User is logged in	User inputs DB host, port, user, pass; system tests connection and saves it.	DB Connection is successfully added to workspace.
Test Connection	System	None	System attempts to connect to the provided DB credentials.	Returns success or failure status.
Update Connection Info	User	DB Connection exists	User edits existing details; system tests and updates the DB record.	DB Connection details are updated.
Delete DB Connection	User	DB Connection exists	User confirms deletion; system removes the connection and cascades associated data.	DB Connection is removed.
Synchronize Meta Schema	User, System Task	DB Connection is active	System runs a background task to extract tables and columns from the target DB.	Meta schema is built as JSON.
Cache Database Metadata	System	Sync is successful	System caches the JSON schema into metadata_cache column in DB.	Schema is ready for LLM context extraction.



Picture 2.4: Conversation and SQL Query Use Case Diagram

Table 2.4: Conversation and SQL Query Use Case Specification

Use Case Name	Actor	Preconditions	Main Flow	Postconditions
Manage Conversations	User	User is logged in	User creates a new chat session linked to a DB connection, or deletes old ones.	A new conversation is created or deleted.
Chat with AI	User	Conversation is active	User sends a prompt or query; system logs the query and triggers LLM processing.	User message is logged and displayed.
Auto-generate SQL	LLM Engine	Prompt is received	System builds context (schema + prompt), calls LLM, and parses the generated SQL.	SQL is generated and returned to UI.
View Conversation History	User	User is logged in	User opens a past conversation; system loads associated query logs.	Previous chat history is displayed.
Rate Response	User	Response generated	User clicks upvote/downvote; system records the rating in feedbacks table.	Rating is saved successfully.
Fix SQL and Send Feedback	User	Response generated	User modifies the generated SQL and submits feedback; system saves corrected_sql.	Feedback data is saved for future training.

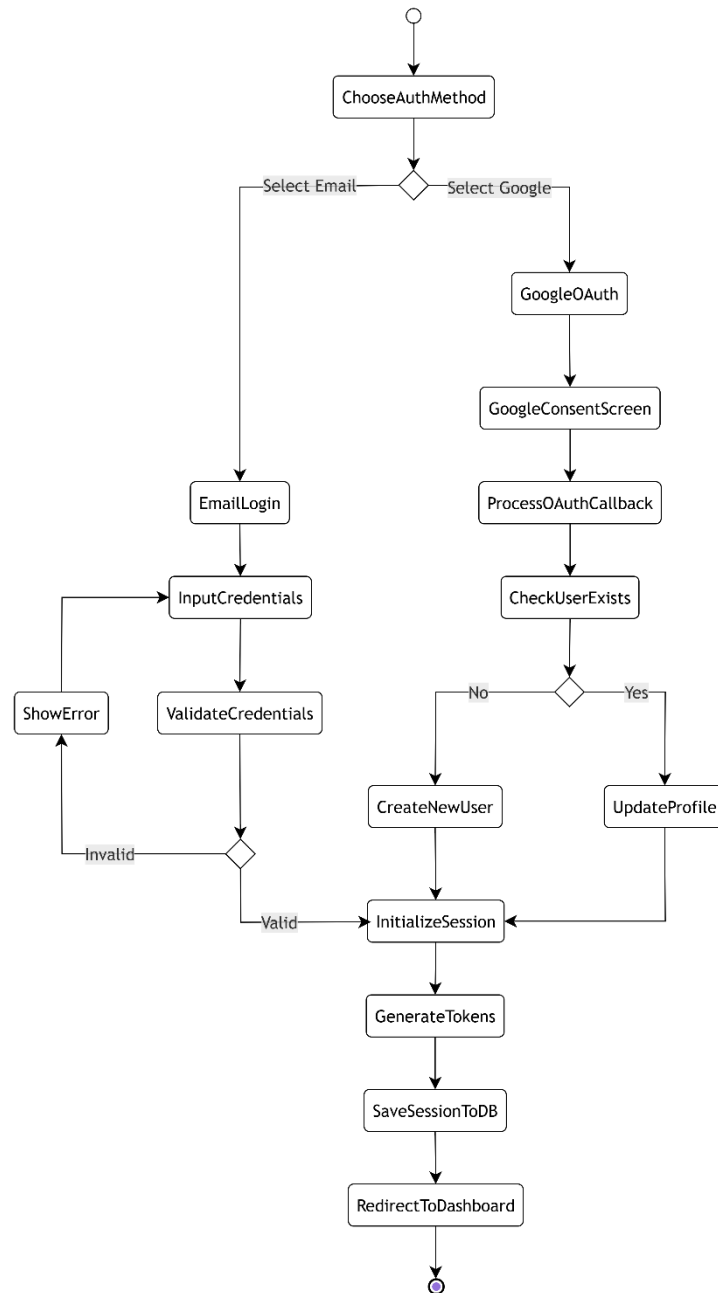


Picture 2.5: Performance Analysis and Optimization Use Case Diagram

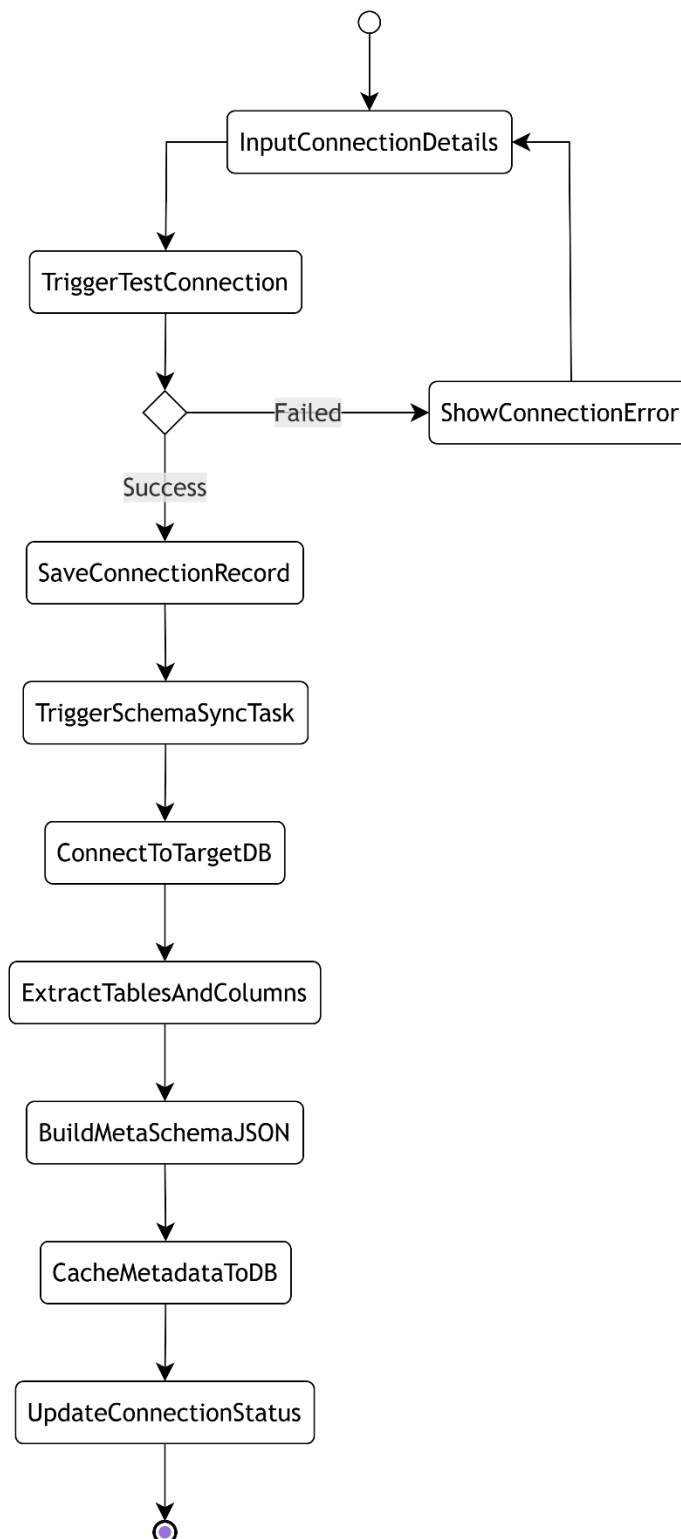
Table 2.5: Performance Analysis and Optimization Use Case Specification

Use Case Name	Actor	Preconditions	Main Flow	Postconditions
Retrieve Explain Plan	User	SQL is provided	User clicks Explain; system connects to DB and runs EXPLAIN query.	Execution plan JSON is retrieved.
Measure Execution Time and Cost	System Task	Plan is retrieved	System extracts cost and actual execution time from the Explain plan.	Performance stats are calculated and saved.
Recommend Index	System Task	Heavy scan detected	System analyzes the plan for sequential scans and suggests index creation.	Index recommendations are displayed.
Optimize SQL via LLM	User, System Task	Plan is retrieved	User clicks Optimize; system triggers background task to prompt LLM for better SQL.	Optimized SQL query is generated.
Run Test in Postgres Sandbox	System Task	SQL is generated	System provisions SQLite/Postgres sandbox, seeds data, and runs the query.	Safe simulation results and metrics are returned.

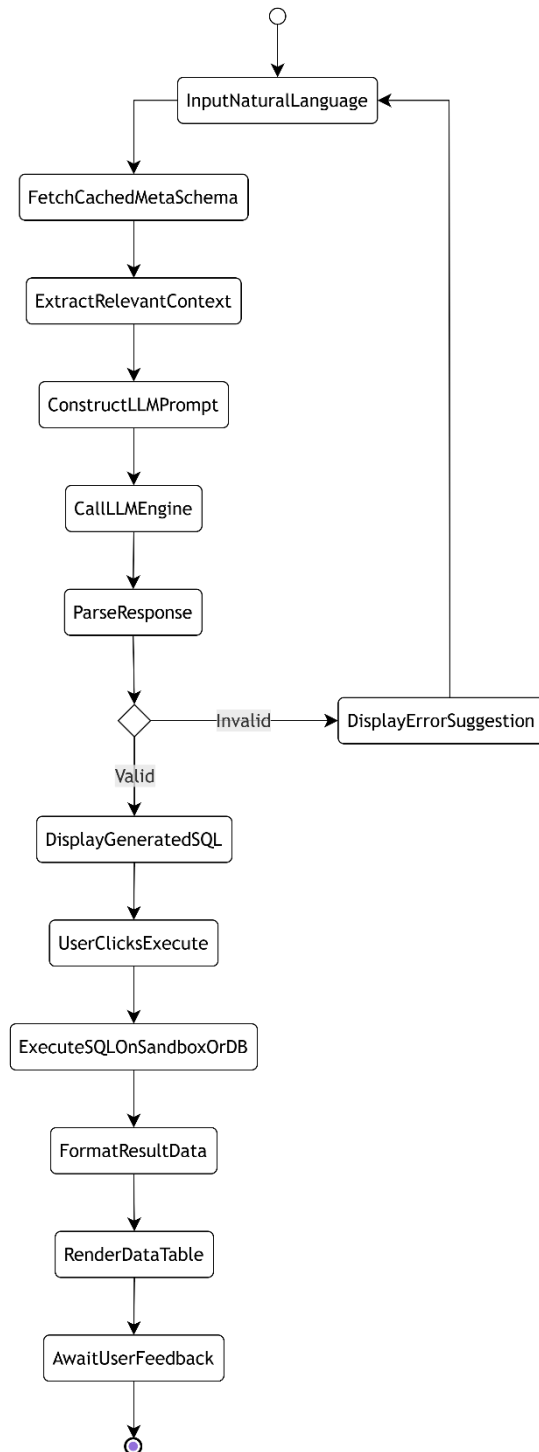
2.4.2. Activity Diagram



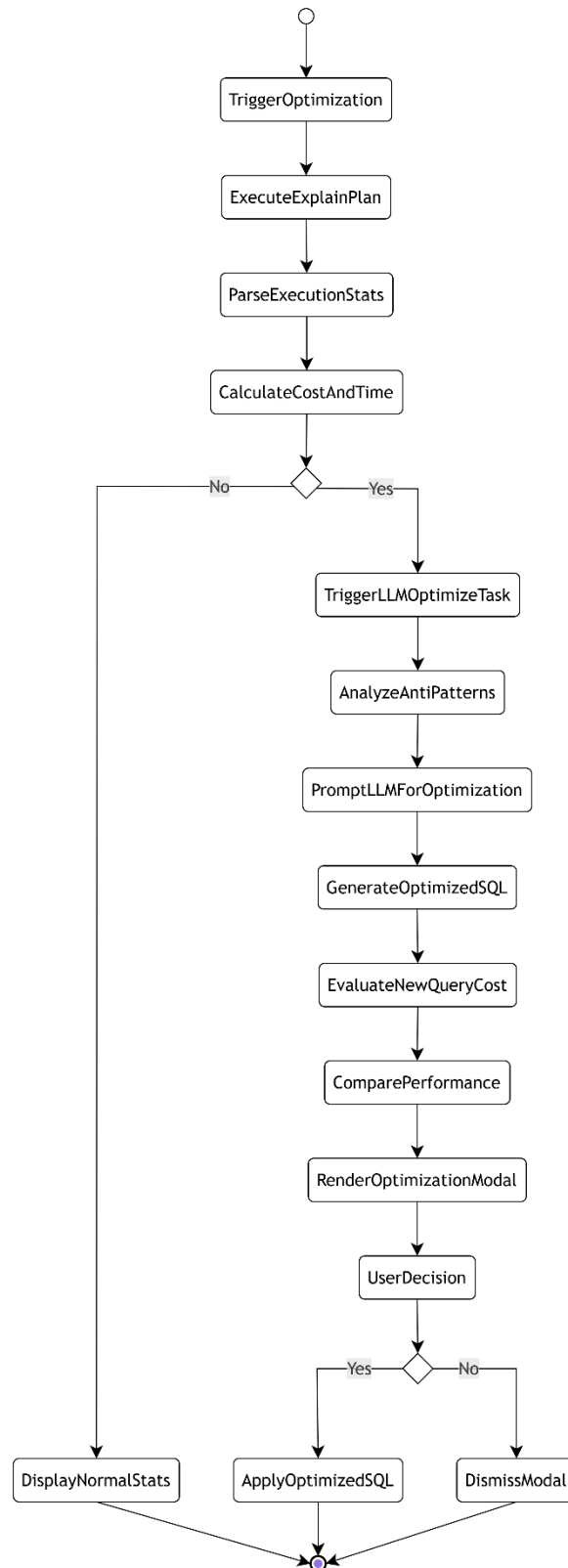
Picture 2.6: Login and Authentication Activity Diagram



Picture 2.7: Add Connection and Schema Synchronization Activity Diagram

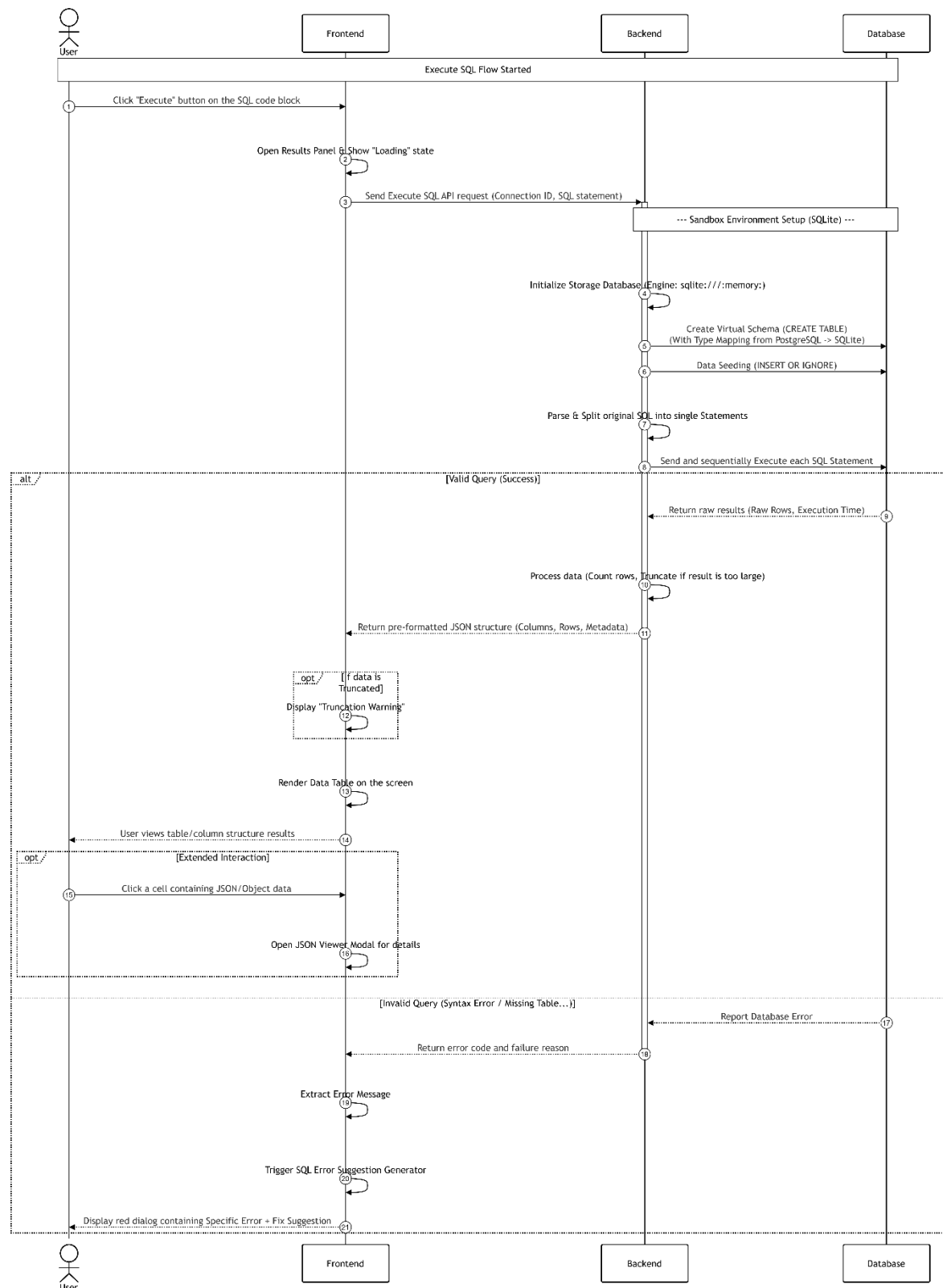


Picture 2.8: Automated SQL Generation (Text-to-SQL) Activity Diagram

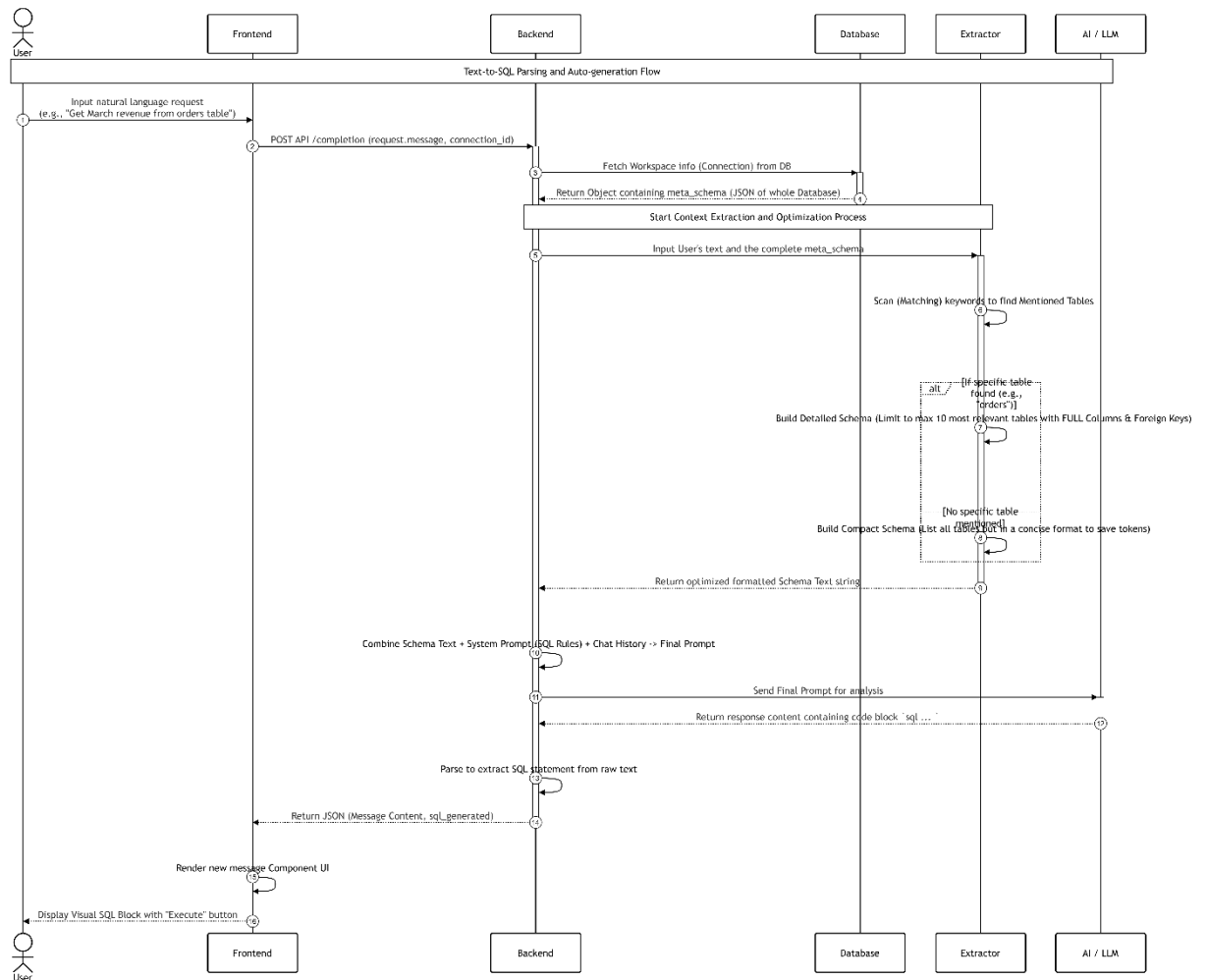


Picture 2.9: Query Analysis and Optimization Activity Diagram

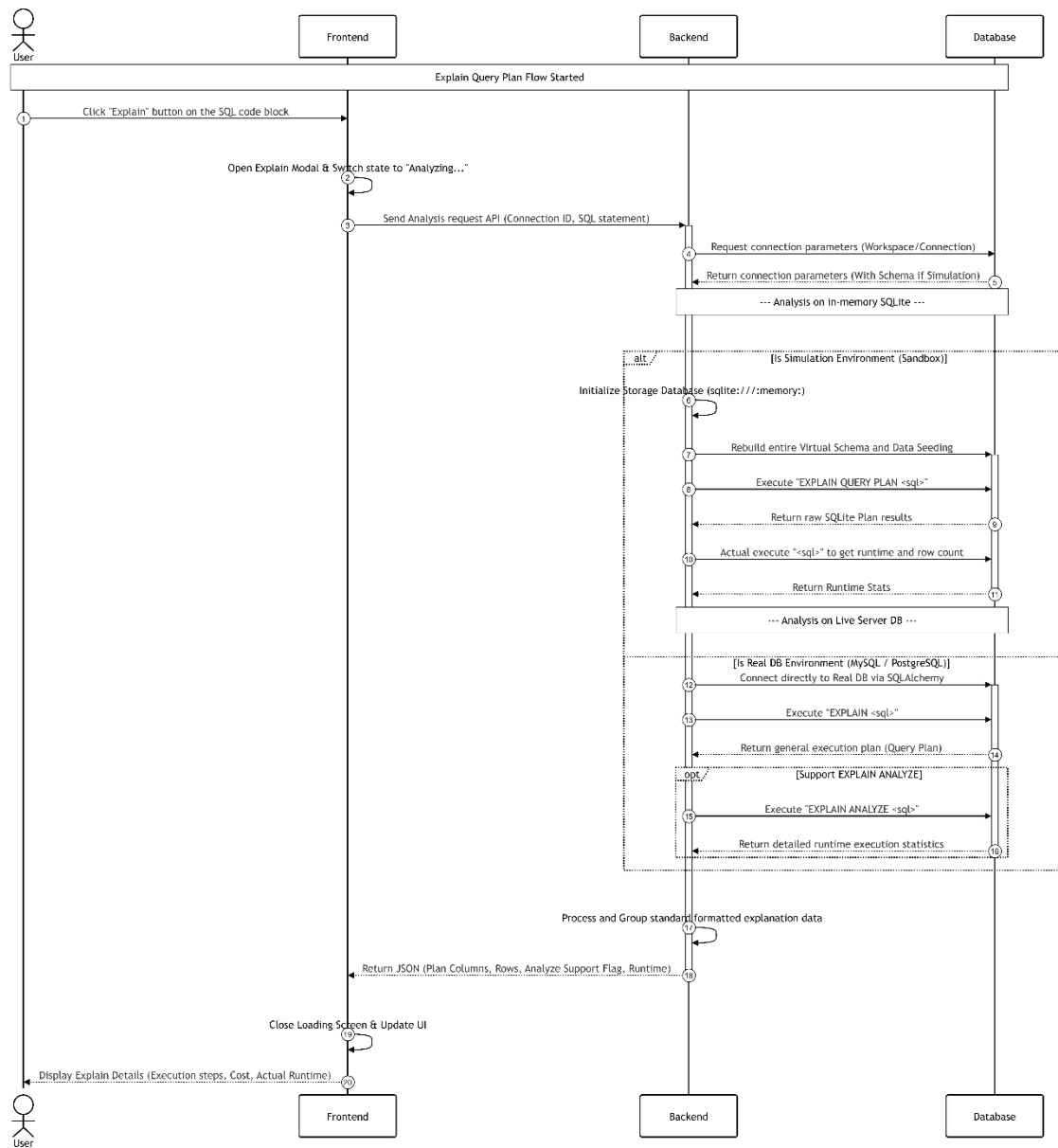
2.4.3. Sequence Diagram



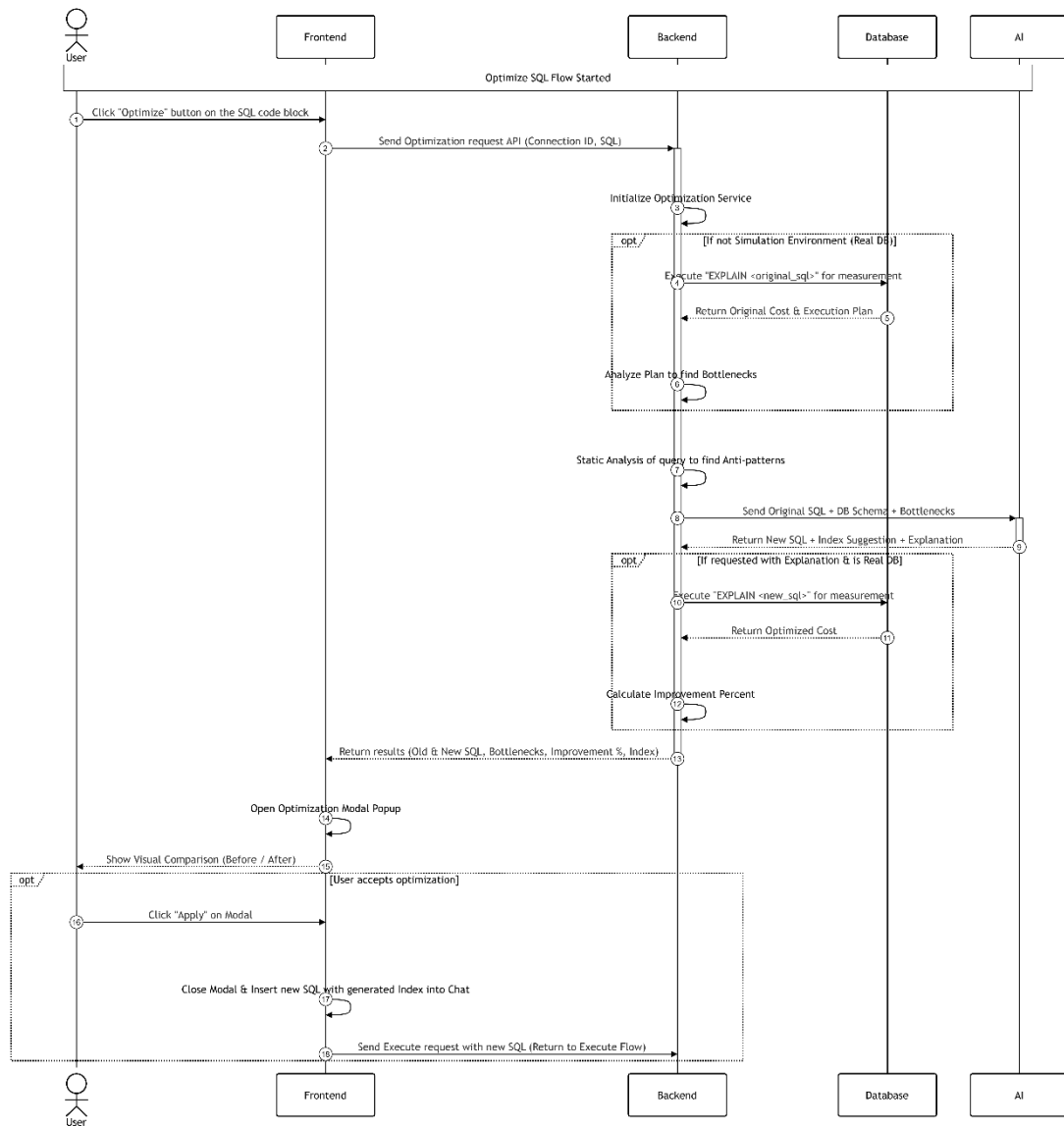
Picture 2.10: SQL Execution on Sandbox Sequence Diagram



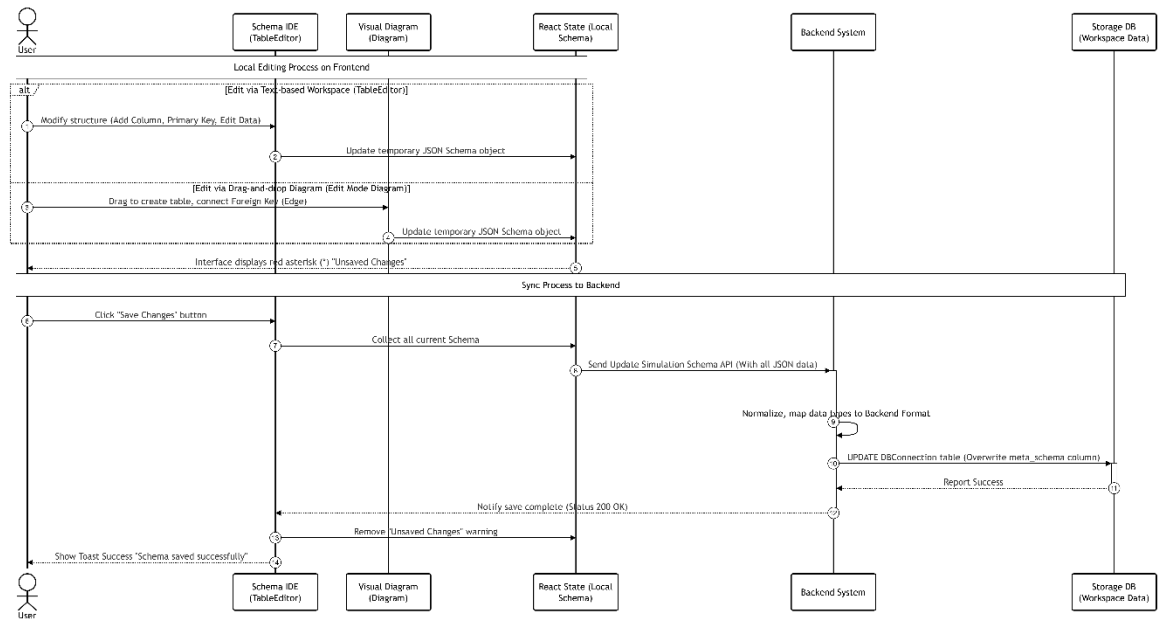
Picture 2.11: Text-to-SQL Parsing and Auto-generation Sequence Diagram



Picture 2.12: Explain Query Plan Sequence Diagram



Picture 2.13: SQL Optimization via LLM Sequence Diagram



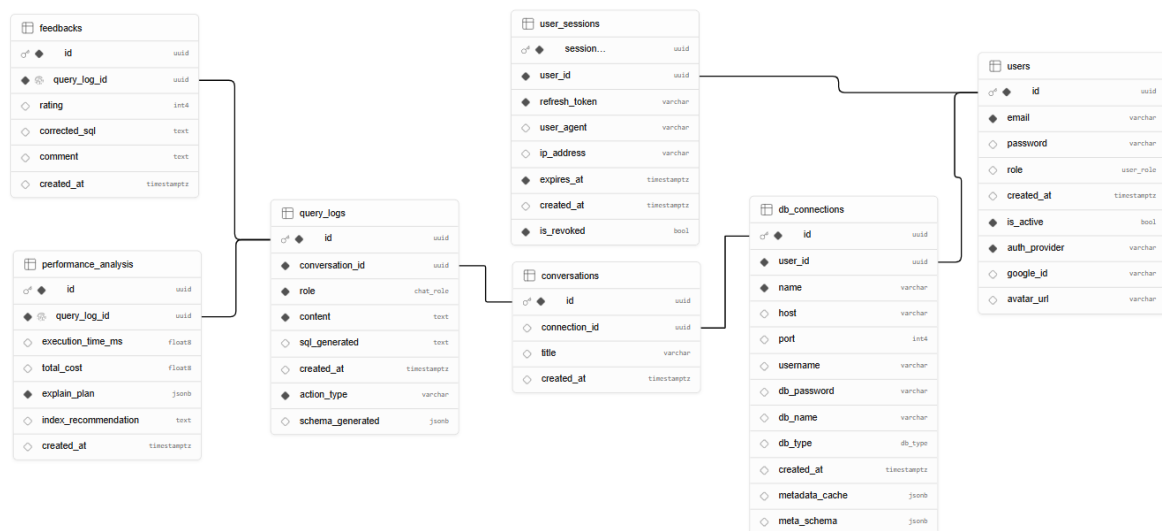
Picture 2.14: Local Schema Editing and Synchronization Sequence Diagram

2.5. Database Design

2.5.1. Overall Diagram

The SQLTuner Database is designed according to a Relational Database model to comprehensively manage data flows from users, system configurations, to AI analysis results. The overall diagram includes main data groups:

- **Identity and Session Group:** Manages users and user_sessions.
- **Connection Configuration Group:** Stores connection info and metadata of target Databases (db_connections).
- **Interaction and Query Generation Group:** Records conversations and details of each Natural Language inference turn to SQL Query (query_logs).
- **Optimization and Feedback Group:** Contains extracted data from Execution Plans to provide optimization recommendations (performance_analysis) and user feedbacks.



Picture 2.15: Database Schema Diagram

2.5.2. Details of Core Data Tables

Table db_connections (Stores connection information and target Database metadata)

This table plays a crucial role in providing context (Schema) for the LLM. It caches the table structures of the target Database.

Table 2.6: db_connections Table Structure Specification

Column Name	Data Type	Constraint	Description
id	UUID	Primary Key	Auto-generated primary key
user_id	UUID	Foreign Key -> users(id)	Linked to user account
name	VARCHAR	Not Null	Mnemonic name of the connection
host, port	VARCHAR, INT		Server address and port (Default 5432)
db_type	db_type	Default 'postgres'	RDBMS classification
metadata_cache	JSONB	Nullable	Cache storage of destination database metadata
meta_schema	JSONB	Default '{}'	Storage of extracted Data Definition Language (DDL) Schema

Table query_logs (Logs details of the Text-to-SQL generation process)

This table records every interaction between the user and the internal AI model.

Table 2.7: query_logs Table Structure Specification

Column Name	Data Type	Constraint	Description
id	UUID	Primary Key	Primary key of the interaction turn
conversation_id	UUID	Foreign Key	Which conversation session it belongs to
role	USER-DEFINED	Not Null	Role (user, system, or assistant)
content	TEXT	Not Null	Natural Language question content
sql_generated	TEXT	Nullable	SQL Query generated by Local LLM
action_type	VARCHAR	Default 'chat'	Action classification
schema_generated	JSONB	Nullable	Trace of schema used as Context Prompt

Table performance_analysis (Stores analysis data and performance optimization recommendations)

Saves the results after the system executes the EXPLAIN command and gets analyzed by the LLM.

Table 2.8: performance_analysis Table Structure Specification

Column Name	Data Type	Constraint	Description
id	UUID	Primary Key	Auto-generated primary key
query_log_id	UUID	Foreign Key, Unique	1-1 link to corresponding SQL generation turn
execution_time_ms	DOUBLE PRECISION	Nullable	Actual Execution Time (ms)
total_cost	DOUBLE PRECISION	Nullable	Estimated computational cost from Database
explain_plan	JSONB	Not Null	Detailed Execution Plan tree in JSON format
index_recommendation	TEXT	Nullable	Optimization recommendation generated by AI

2.6. System Construction

2.6.1. Project Folder Structure Organization

The system source code is organized into a modular layered architecture, isolating business scopes and optimizing resource management when working with LLMs:

2.6.2. Database Simulation (Sandbox Environment) Mechanism Design

The Database Simulation / Sandbox component is built deep inside the Service layer of the FastAPI Backend to safely and optimally manage Query Execution lifecycles. The architecture of this mechanism follows a Transaction-based Sandbox model:

- **Isolated Session Init:** When the evaluation process starts, the system does not use the main application's Connection Pool. Instead, an independent Connection Session with PostgreSQL is initialized via the SQLAlchemy library.
- **Protection via Transaction & Savepoint:** Before executing any AI-generated SQL Queries, the system automatically opens a Database Transaction (BEGIN) and marks a SAVEPOINT.
- **Execution & Catching:** The Predicted SQL is executed in this safe zone. If the system detects DML/DDI queries altering data state rather than just querying (SELECT), it is caught via Exception Handling.
- **Rollback & Teardown:** Whether the query succeeds returning a Result Set or fails (Runtime/Syntax Error), the system triggers a ROLLBACK to undo all temporary changes, ensuring the test Database retains a pure Zero-mutation state for thousands of subsequent Queries.

2.6.3. Evaluation Pipeline Workflow

The execution workflow of the Automated Evaluation Framework is strictly sequential, ensuring high precision and tight management of internal hardware resources:

- **Step 1: Initialization:** Users select the dataset (e.g., Spider) and configure Local LLM parameters to be evaluated on the Frontend. The Backend reads the data list from the Benchmark config file and creates pending status records.
- **Step 2: Schema Linking & Prompt Building:** For each question in the queue, the system automatically extracts the meta_schema (DDL) to attach to the sample Prompt, ensuring the model gets adequate column and table info.
- **Step 3: Asynchronous Inference:** The inference process asynchronously calls Local LLMs deployed on internal servers. Using Queue Management and strict Batch Size control, the system optimally coordinates CPU/GPU compute resources, completely preventing Out of Memory (OOM) situations during high-intensity Batch Inference. The Generated SQL is stored in working memory.

- **Step 4: Execution in Database Simulation Sandbox:**
 - Activates the Sandbox module, opening an isolated Transaction on PostgreSQL.
 - Executes the newly generated Predicted SQL to fetch the Result Set R_{pred} . If there's a syntax error, logs it and assigns Execution Accuracy (EX) = 0.
 - Executes the Ground Truth SQL to get the standard Result Set R_{true} .
 - Performs Deep Comparison between R_{pred} and R_{true} . If fully matched, EX = 1, otherwise EX = 0.
- **Step 5: Exact Match Checking:** Normalizes both SQL Query text strings and performs direct string comparison to assign Exact Match (EM) score (0 or 1).
- **Step 6: Logging & Finalization:** Detailed results of each question are saved (query_logs). The system computes the final average metrics, updates the test run status to completed, and signals the Frontend to display analytical charts.

2.7. Conclusion of Chapter 2

Chapter 2 fully detailed the overall analysis and design landscape of the SQLTuner project. Starting from pinpointing practical bottlenecks of the Text-to-SQL Problem, it delineated functional modules corresponding to specific actors. Core focus was on clarifying the Pipeline Workflow architecture fully leveraging Local LLMs, ensuring Data Privacy, and infrastructure autonomy.

CHAPTER 3: DEPLOYMENT AND EVALUATION RESULTS

3.1. Deployment Model

3.1.1. Deployment Environment

The entire SQLTuner system is containerized using Docker to ensure independence, easy maintenance, and operational environment synchronization.

- **Consistency:** Using docker-compose sets up internal networks and storage volumes. Code behaves exactly the same on Local development machines as on Cloud computing servers.
- **Local LLM Deployment:** The AI processing module is segregated into an independent service within the container architecture, allowing transparent resource management (CPU limits, RAM/VRAM allocation) preventing AI models from monopolizing API service resources.

3.1.2. Tools Used

- **Visual Studio Code (VS Code):** The main Integrated Development Environment (IDE) used to write Python (Backend) and TypeScript (Frontend), along with real-time Docker log monitoring.
- **Swagger UI:** Used to document and visualize API endpoints, allowing for direct API Testing in the browser.
- **PgAdmin4:** Graphical Database management platform used to monitor configuration data, review evaluation_logs, and supervise test Database states (like Spider, BIRD datasets).

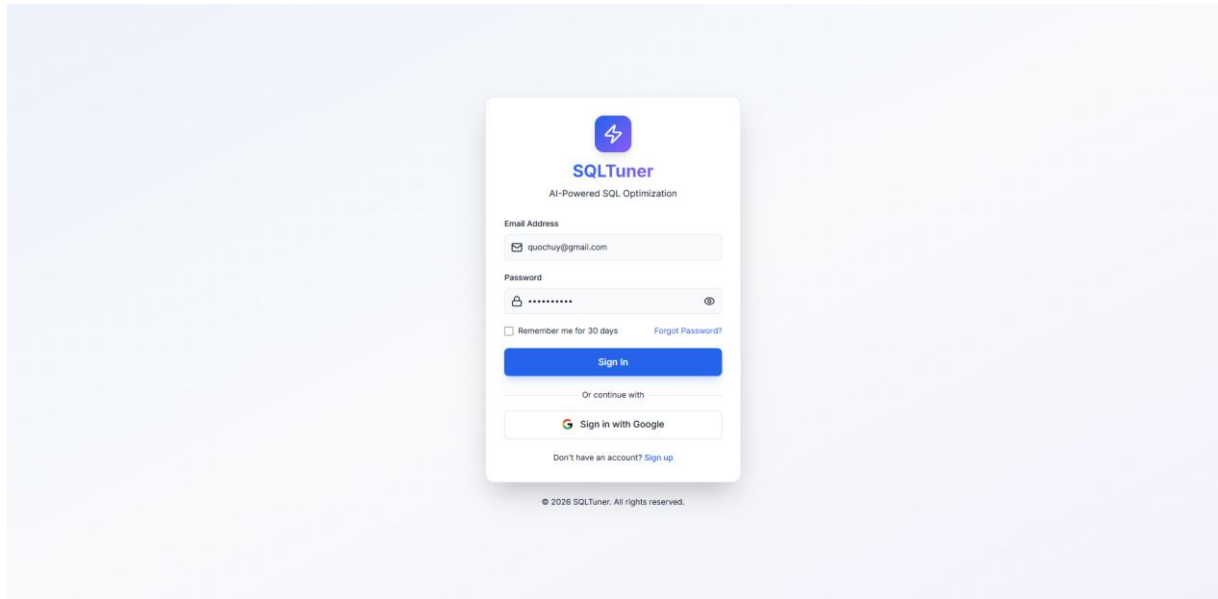
3.1.3. System Configuration

Configurations focus on defining model parameters and hyper-parameters through the environment file:

- **Model Selection:** Pointing to open-source models specialized for programming (e.g., Qwen-2.5-Coder, Llama-3) hosted internally.
- **Generation Parameters:** TEMPERATURE set low (0.1 - 0.2) to eliminate hallucination, enforcing strict logic structure in generated SQL Queries. MAX_TOKENS capped to save compute resources.
- **Batch Processing Parameters:** Configured BATCH_SIZE and MAX_CONCURRENT_REQUESTS to fit hardware VRAM limits for Batch Inference execution.

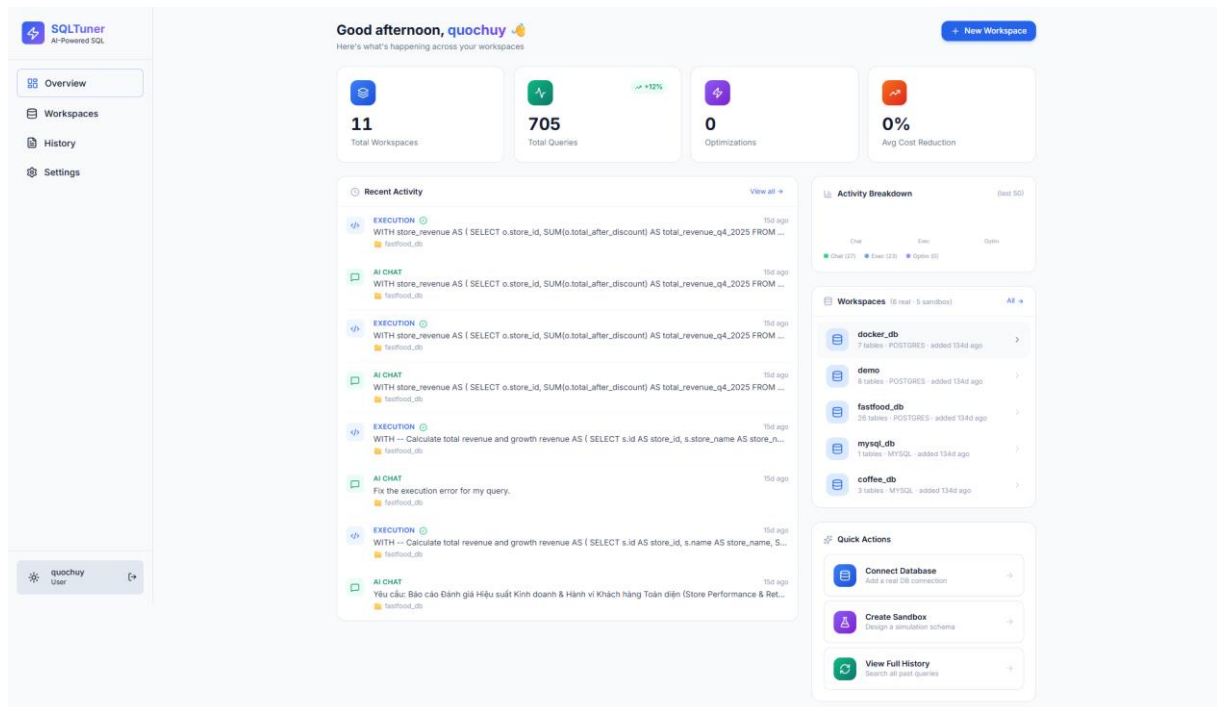
3.2. System User Interface

3.2.1. User Interface



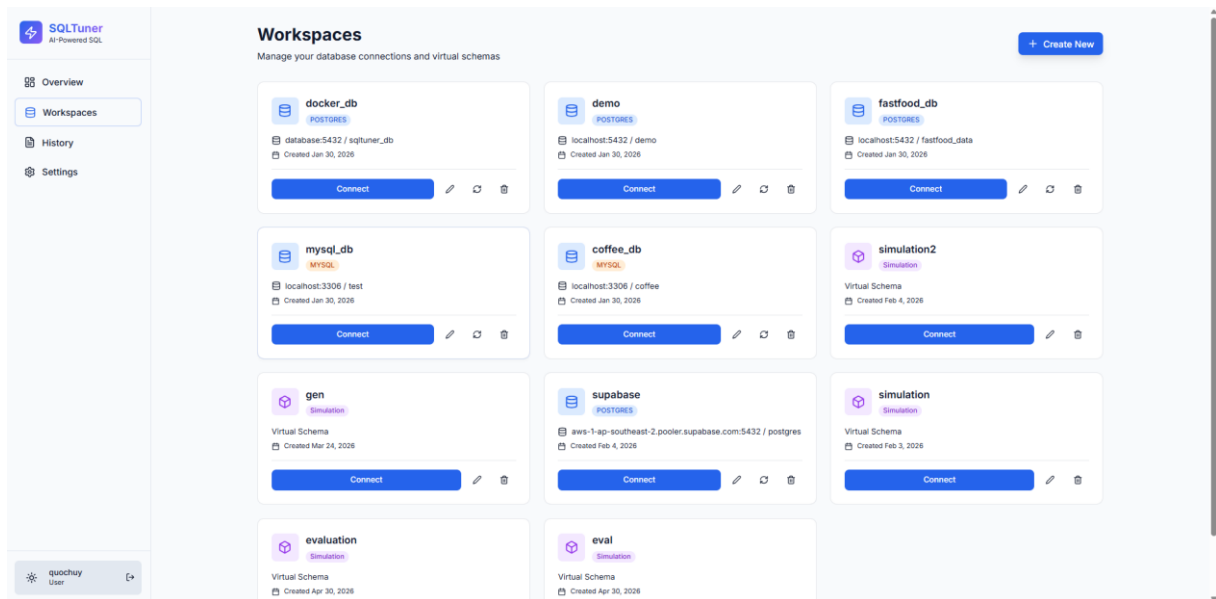
Picture 3.1: User Login and Authentication Interface

This interface serves as the primary secure gateway to the SQLTuner platform. It provides users with two flexible authentication options: a traditional login utilizing a registered email and password combination, or a convenient Single Sign-On (SSO) approach using Google accounts for faster access.



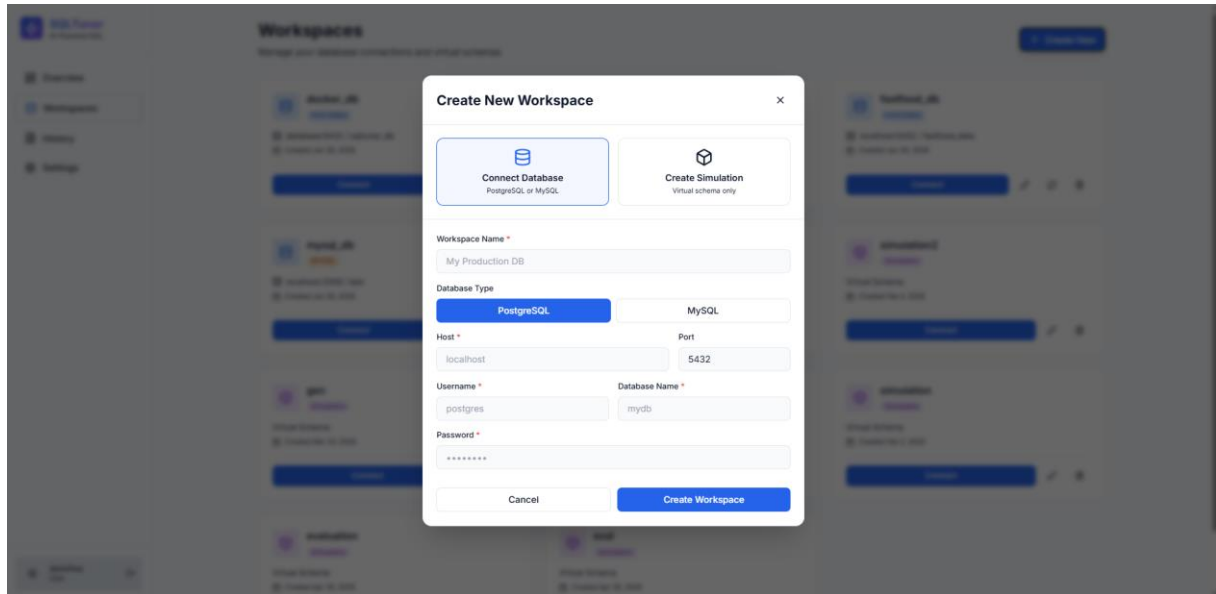
Picture 3.2: Main Overview Dashboard and Activity Tracking

Upon successful authentication, users are directed to the Main Overview Dashboard. This centralized hub provides a comprehensive view of the user's workflow, displaying key metrics such as total queries and active workspaces. It also allows users to track their recent AI interactions, monitor query execution history, and quickly access quick actions like connecting to a new database or creating a sandbox.

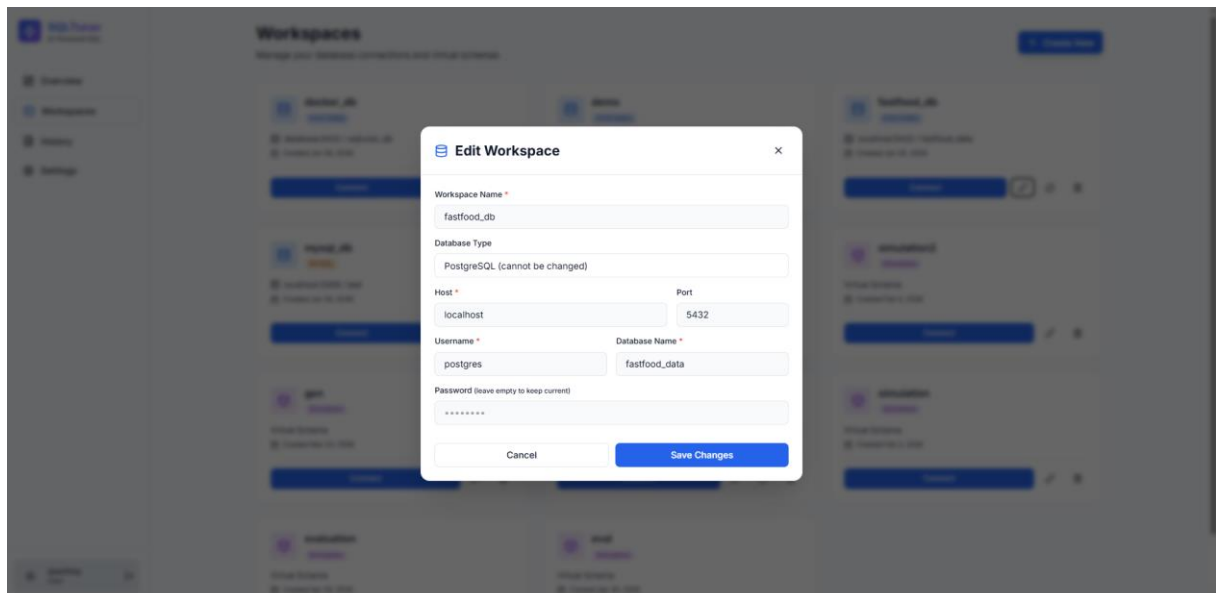


Picture 3.3: Workspaces and Database Connection Management Dashboard

The Workspaces interface is designed for comprehensive management of both real database connections (such as PostgreSQL and MySQL) and virtual simulation schemas. Each workspace is displayed as an individual card detailing the database type, connection string, and creation date. From this centralized dashboard, users can seamlessly connect to initiate a chat session, edit credentials, manually trigger schema synchronization, or delete existing environments.

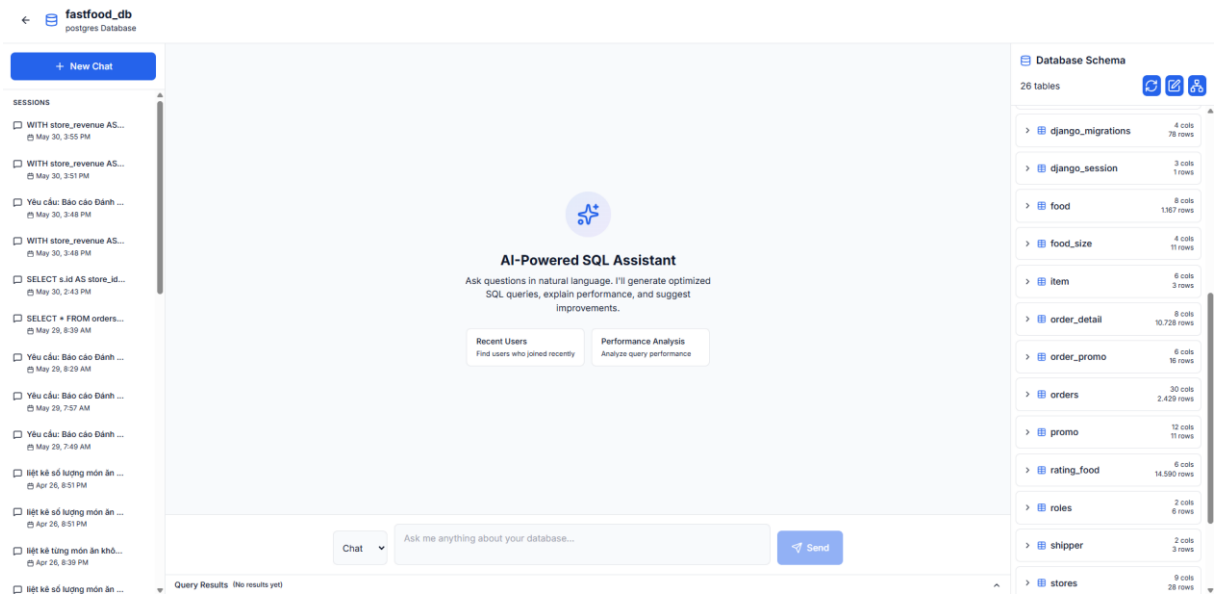


Picture 3.4: Create New Workspace Configuration Modal

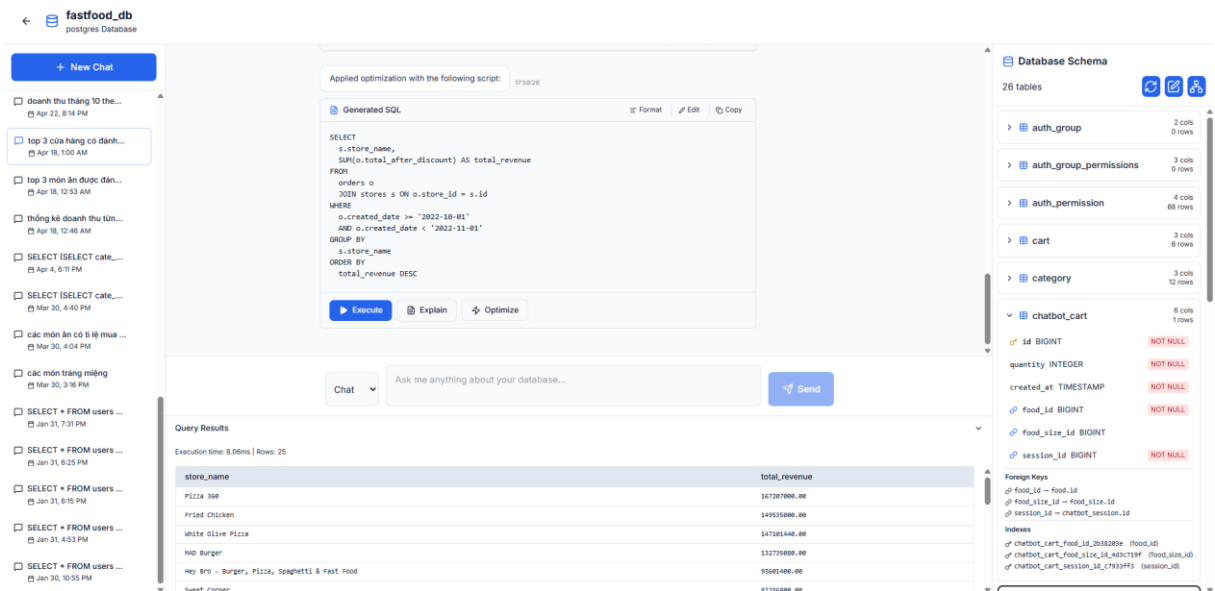


Picture 3.5: Edit Workspace and Synchronization Modal

These configuration modals provide users with an intuitive interface to set up and manage database environments. Users can easily create a new workspace by selecting the database type (PostgreSQL or MySQL) and inputting the necessary credentials. Additionally, users can update existing workspace information. Upon saving, the system securely connects to the target database and triggers a background task to synchronize the metadata schema, ensuring the AI model has the most up-to-date context for accurate SQL generation.

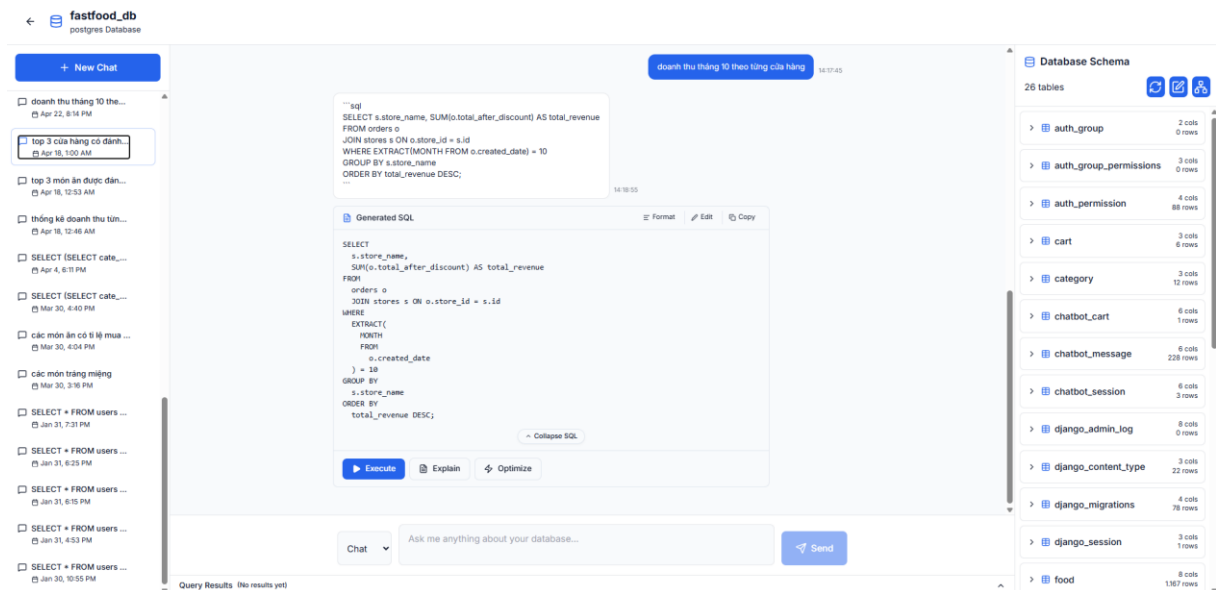


Picture 3.6: Intelligent Text-to-SQL Chat Interface Layout



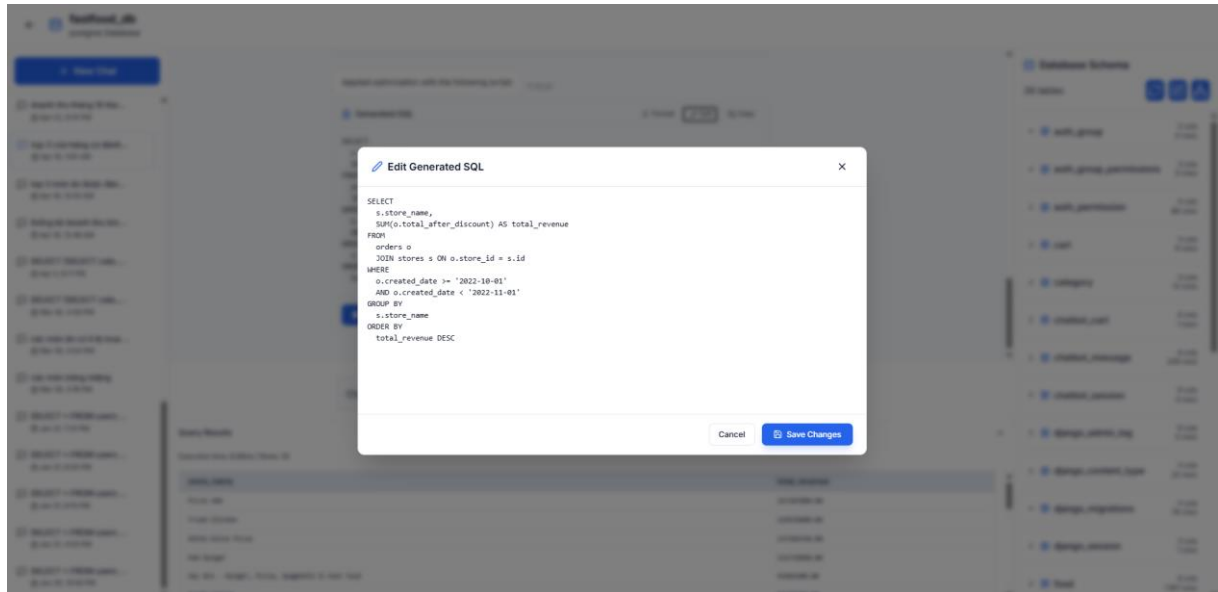
Picture 3.7: SQL Generation, Execution Results, and Schema Inspector

These images present the core Intelligent Text-to-SQL Chat interface. The layout is strategically divided into three main sections: a left sidebar for managing conversation history, a central working area for natural language prompting and SQL generation, and a right sidebar displaying the synchronized Database Schema. After the AI generates the SQL statement, users can trigger the 'Execute' action, and the fetched data is cleanly rendered in the bottom Query Results panel. Furthermore, the interactive Database Schema panel allows users to expand specific tables to inspect detailed metadata, including column data types, indexes, and foreign key relationships, providing essential context during the query formulation process.



Picture 3.9: User-AI Chat Interaction for SQL Generation

This image highlights the direct chat interaction between the user and the AI assistant. Users can seamlessly type their database requests in natural language, just like a standard conversation. In response, the system dynamically analyzes the prompt against the database schema and replies with the corresponding SQL script. This conversational format makes database querying highly intuitive and allows users to iteratively refine their requests based on the AI's feedback.



Picture 3.10: Edit Generated SQL Modal for Manual Refinement

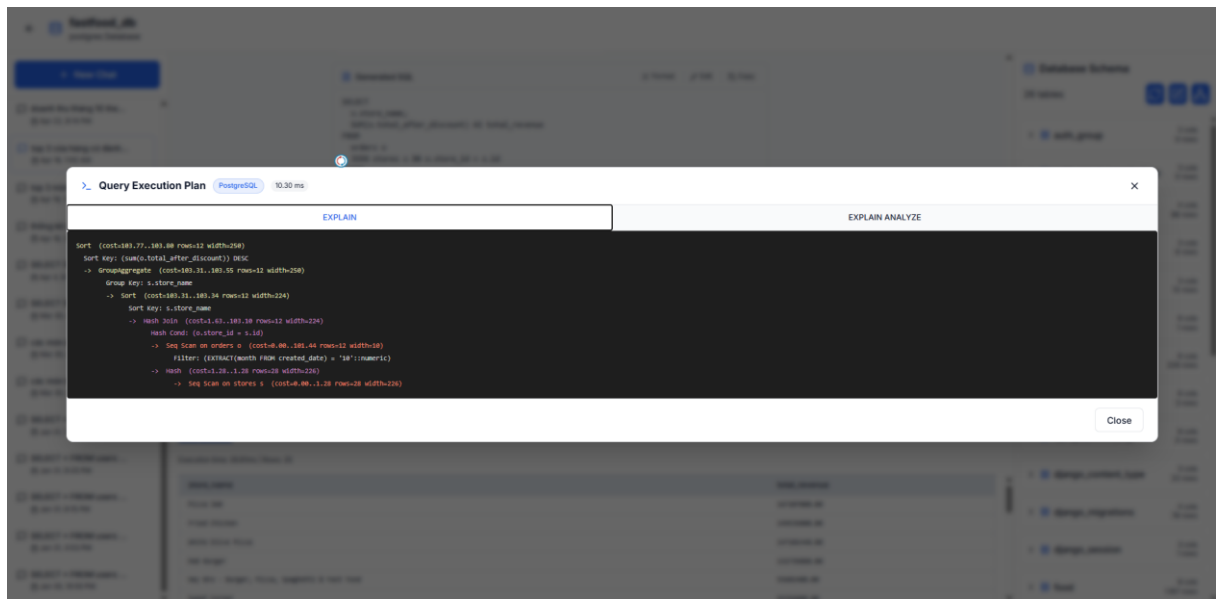
While the underlying Large Language Model provides highly accurate initial queries, this modal empowers users to manually edit the generated SQL script. Developers and DBAs can use this feature to fine-tune the logic, adjust specific filtering conditions, or correct any minor contextual discrepancies. By allowing these manual modifications before execution, the system ensures that the final query strictly aligns with the user's precise intent and yields the exact desired result set.

The screenshot displays the SQLTuner web interface for a database named 'fastfood_db'. On the left, a sidebar lists recent chat queries. The main area features a 'Generated SQL' panel with a query to calculate total revenue by store name, filtered by a specific month and date range. Below the query are buttons for 'Execute', 'Explain', and 'Optimize'. A chat input field at the bottom allows users to ask questions about the database. On the right, a 'Database Schema' panel lists 26 tables with their respective column and row counts. The 'Query Results' panel at the bottom shows the execution output as a table with 25 rows.

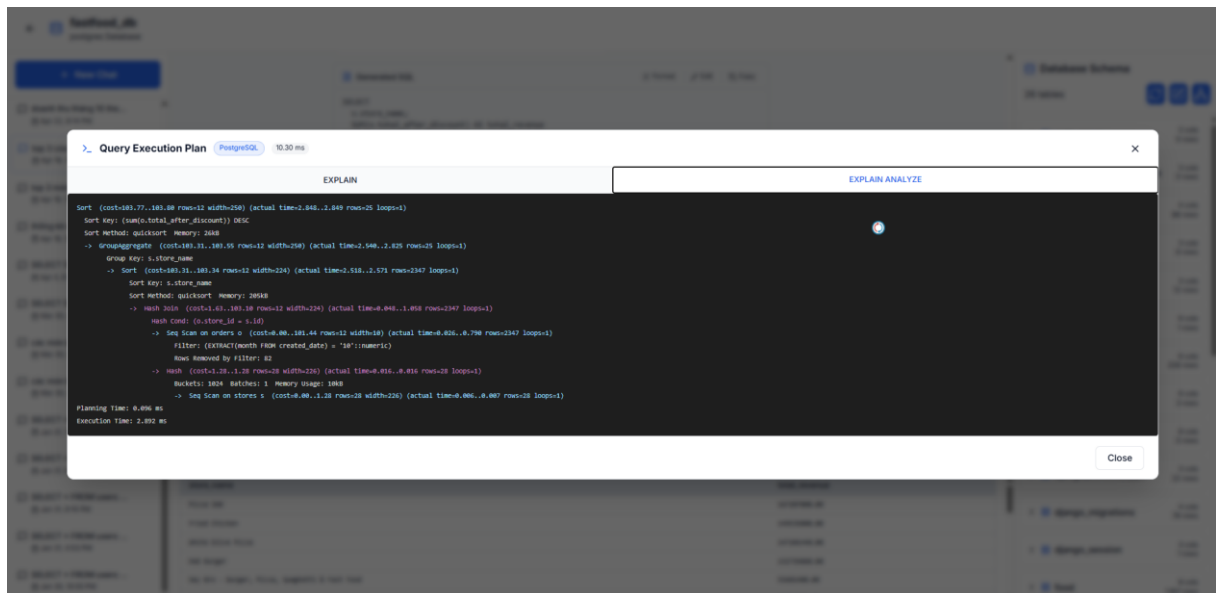
store_name	total_revenue
Pizza 360	167267000.00
Fried Chicken	149532000.00
White Olive Pizza	1471821400.00
McD burger	132735000.00
Hey Bro - Burger, Pizza, spaghetti & fast food	93605400.00
Sweet Corner	82256800.00

Picture 3.11: SQL Query Execution and Result Set Rendering

This interface demonstrates the direct execution capability of the SQLTuner system. Upon clicking the 'Execute' button, the generated or manually refined SQL script is executed against the target database. The system seamlessly retrieves the response and renders the tabular data in the 'Query Results' panel, displaying the execution time and row counts. This streamlined workflow eliminates the need to manually copy SQL statements into external database clients to verify the returned Result Set.

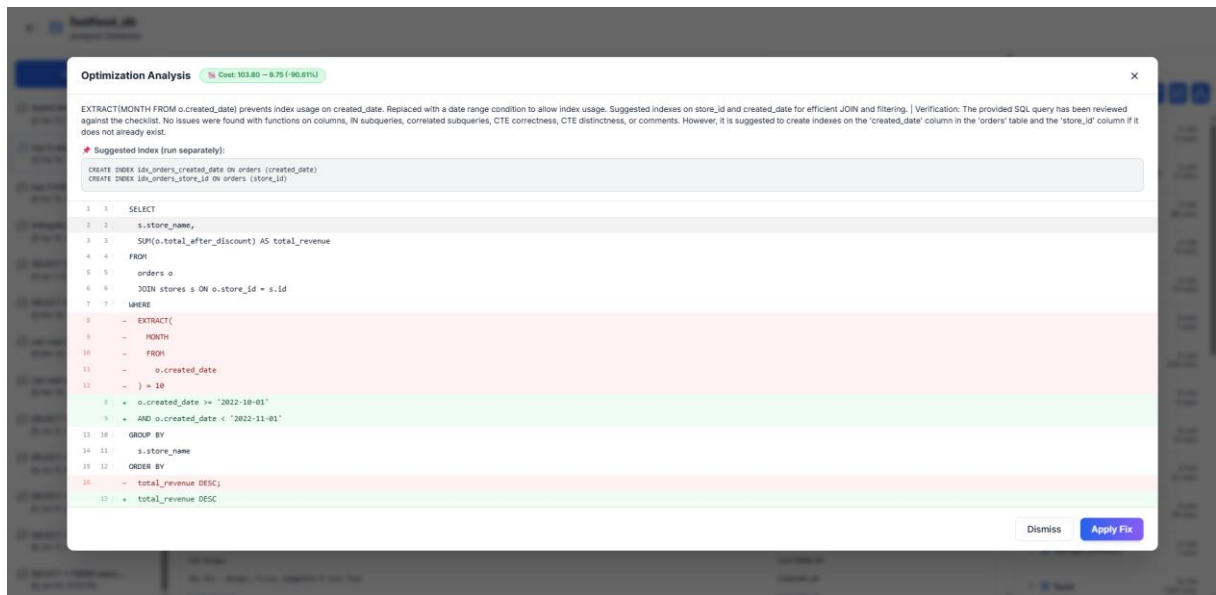


Picture 3.12: Query Execution Plan Analysis Modal (EXPLAIN)



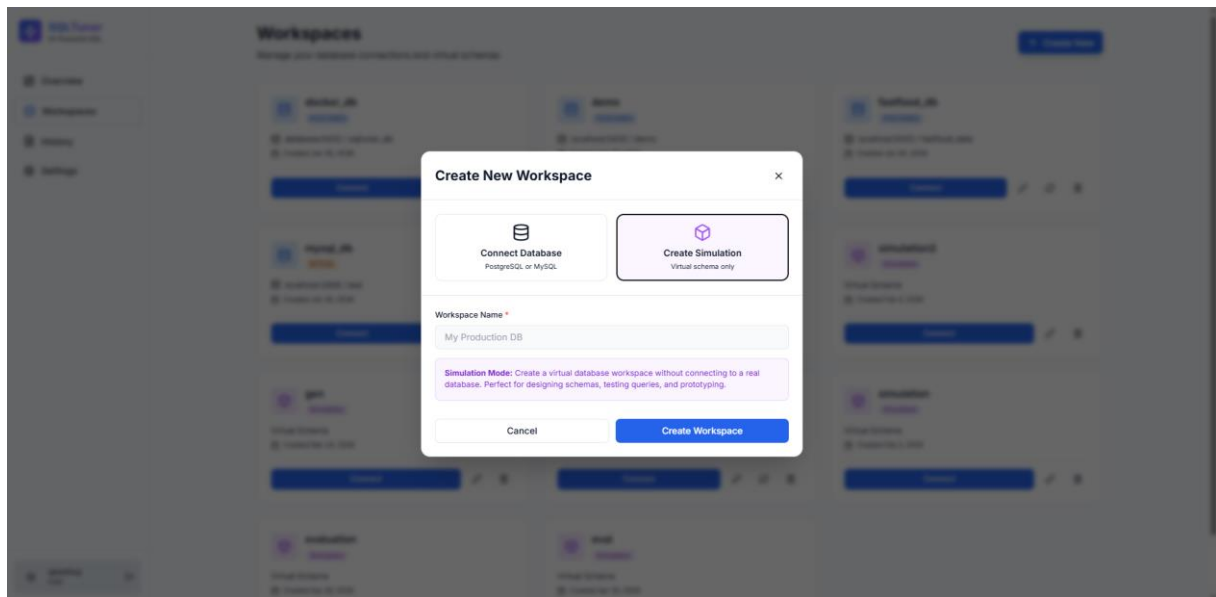
Picture 3.13: Query Execution Plan Analysis Modal (EXPLAIN ANALYZE)

This interface showcases the system's deep integration with database performance tuning mechanics. By clicking the 'Explain' button, the system automatically triggers both EXPLAIN and EXPLAIN ANALYZE commands for the active SQL script. The modal then presents the detailed execution plan returned directly from the PostgreSQL database. Users can seamlessly toggle between the standard 'Explain' tab to review estimated computational costs and the 'Explain Analyze' tab to inspect actual execution times, loops, and detailed node operations (such as Hash Joins and Sequential Scans). This level of insight is crucial for diagnosing structural bottlenecks in complex queries.

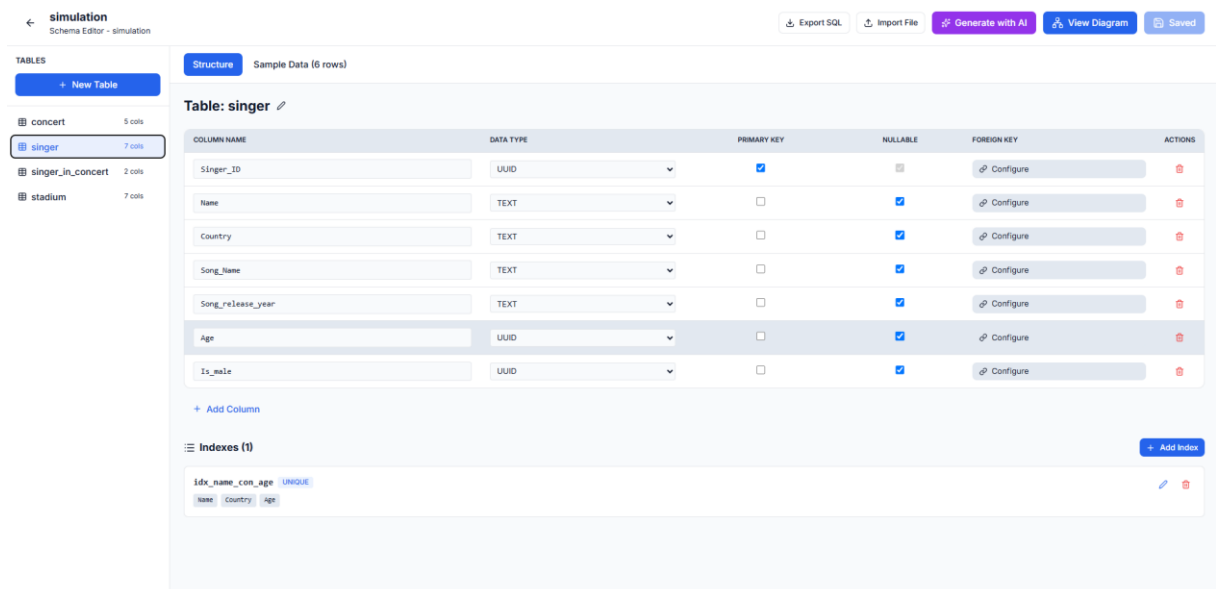


Picture 3.14: AI-Driven Optimization Analysis and Refactoring Proposal

This image highlights the advanced performance tuning capability of the SQLTuner system. Triggered by the 'Optimize' button, the AI module deeply analyzes the query's execution plan to identify computational bottlenecks. The system then proposes a refactored, highly optimized SQL script, which is presented in a clear visual diff format to easily compare the original and improved code lines. Furthermore, the modal explains the optimization rationale (such as replacing functions on columns with date ranges to utilize indexes), provides actionable CREATE INDEX recommendations, and quantifies the exact performance gain by displaying the estimated cost reduction percentage. Users can instantly adopt these enhancements by clicking the 'Apply Fix' button.

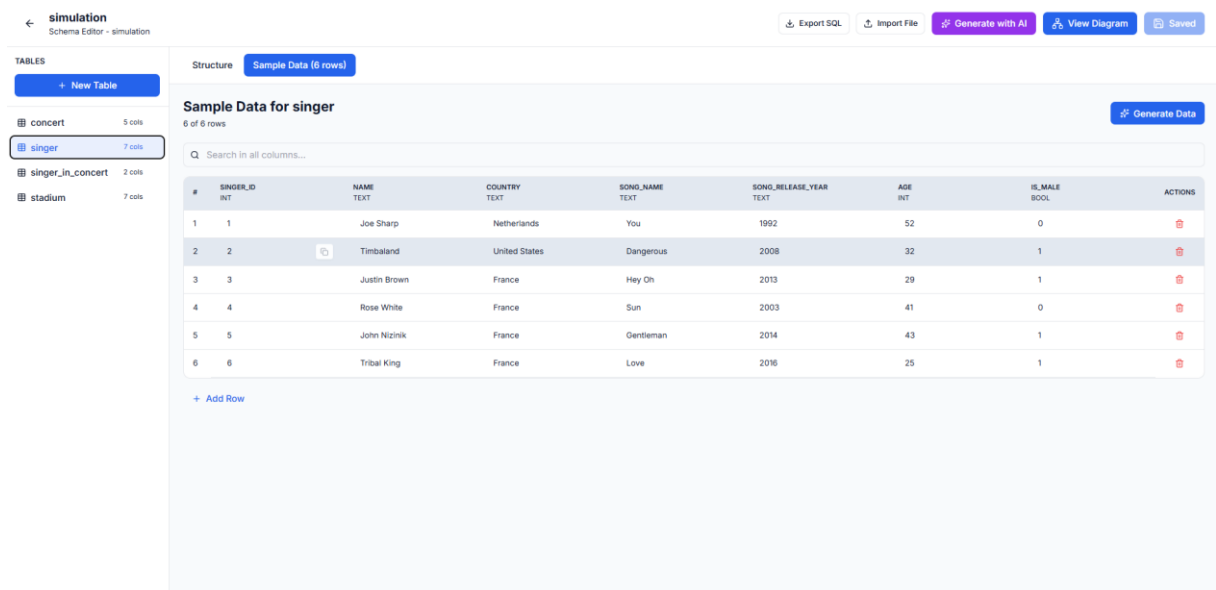


Picture 3.15: Creating a Virtual Simulation Workspace



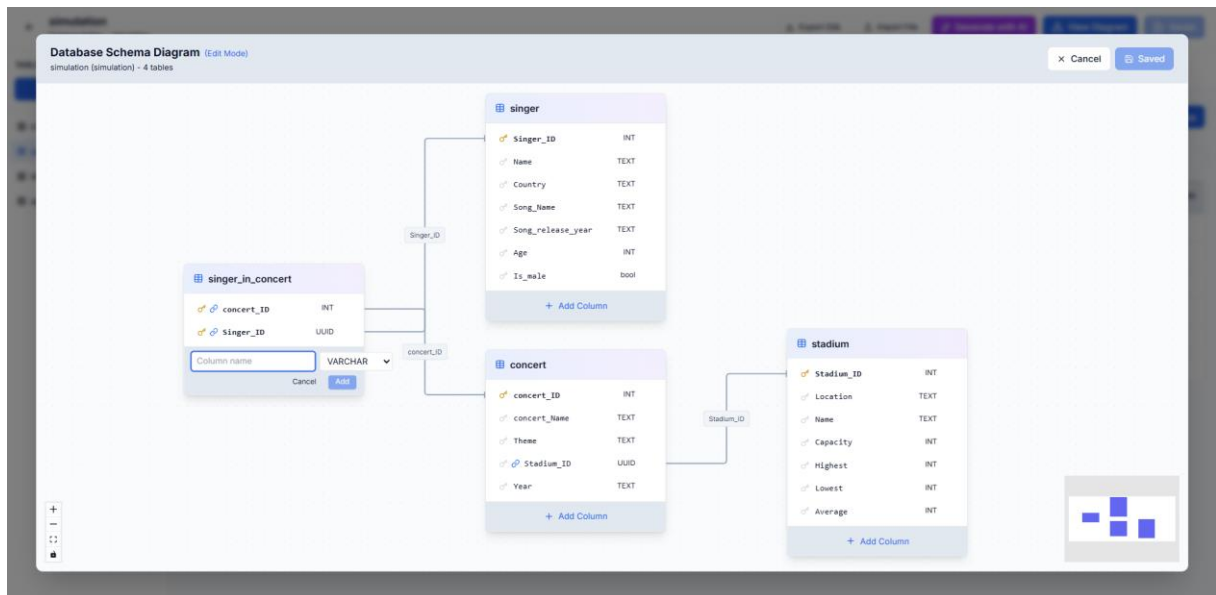
Picture 3.16: Manual Database Schema Editor for Simulation

To ensure absolute safety when automatically executing AI-generated source code and to facilitate prototyping, the system offers a 'Simulation Mode.' As shown, users can create a virtual database workspace without requiring a live database connection. Once created, users access the 'Schema Editor,' a comprehensive manual configuration interface. Here, developers can manually construct the database structure from scratch by defining tables, assigning data types, configuring primary/foreign keys, and adding indexes. This zero-mutation sandbox environment is perfect for designing schemas, safely testing complex queries, and evaluating models without risking data loss in real production environments.



Picture 3.17: Sample Data Generation and Management for Simulation

To complement the virtual schema design, the Simulation Mode also allows users to populate their tables with mock data. By navigating to the 'Sample Data' tab, users can manually insert rows or utilize the built-in data generation tools to automatically fill the table with realistic dummy records. This crucial capability ensures that users can actually execute their generated or refined SQL queries within the sandbox, allowing them to thoroughly verify the query logic and validate the resulting output before applying any operations to a real database.



Picture 3.18: Interactive Visual Diagram Editor for Schema Design

In addition to the standard table-based editor, the Simulation Mode provides an interactive visual diagram interface (Edit Mode). This feature allows users to intuitively design their database architecture using a drag-and-drop canvas. Users can quickly add new tables, define columns directly within the diagram blocks, and seamlessly establish complex foreign key relationships by visually drawing connections between tables. This graphical approach offers a more comprehensive and holistic view of the data structure, significantly accelerating the prototyping and design phase.

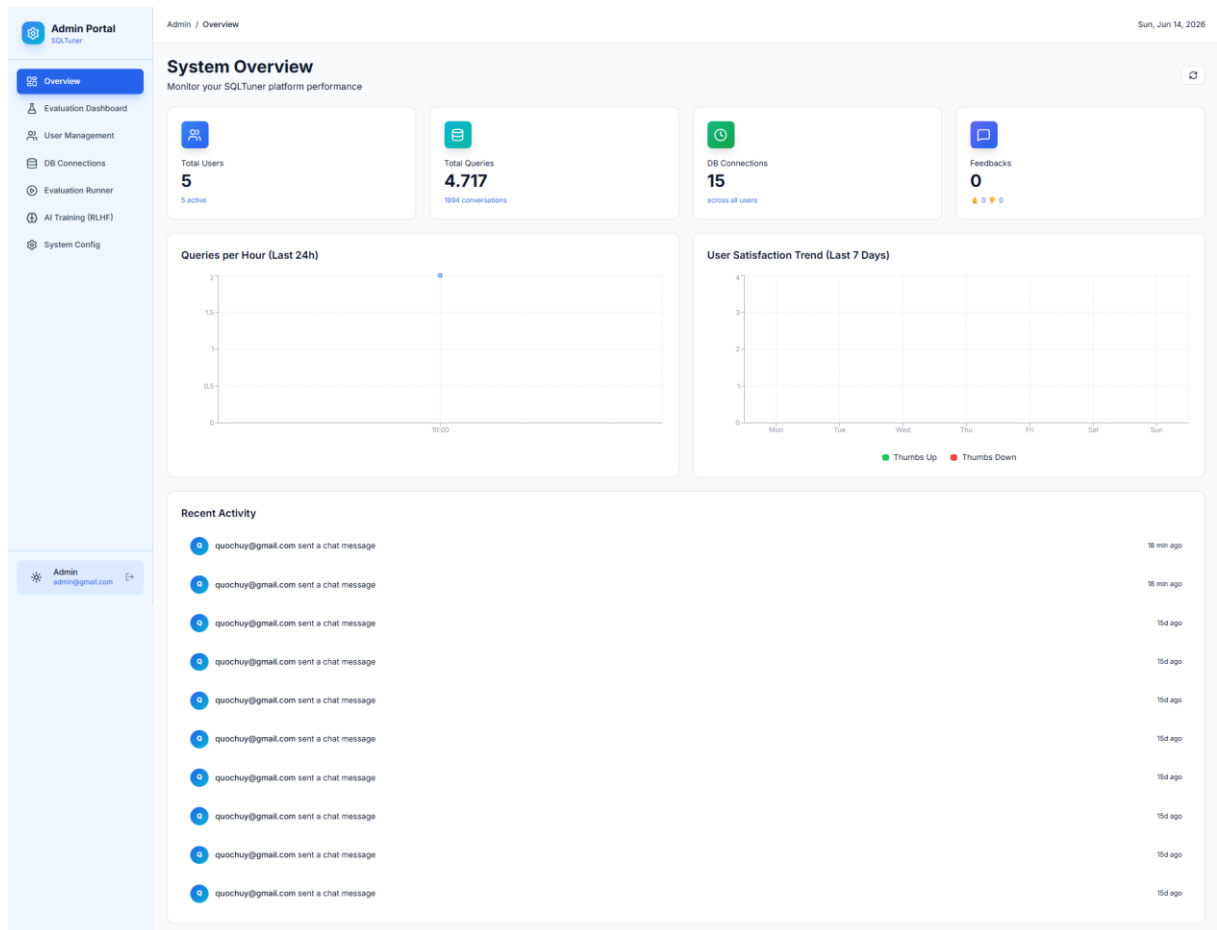
The screenshot shows the 'Global History' section of the SQLTuner application. It features a sidebar with navigation options: Overview, Workspaces, History (selected), and Settings. The main area displays a table of activities with columns for Timestamp, Workspace, Type, Activity, Result, and Actions. The table lists various SQL execution and chat prompts from May 30, 2023, with results ranging from 'Success' to '-'. A search bar at the top allows filtering by SQL content or prompt. Dropdown menus for 'All Workspaces' and 'All Activities' are also present.

TIMESTAMP	WORKSPACE	TYPE	ACTIVITY	RESULT	ACTIONS
May 30 03:55 PM	fastfood_db postgres	Execution	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	Success	View
May 30 03:55 PM	fastfood_db postgres	Chat	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	-	View
May 30 03:51 PM	fastfood_db postgres	Execution	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	Success	View
May 30 03:51 PM	fastfood_db postgres	Chat	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	-	View
May 30 03:50 PM	fastfood_db postgres	Execution	WITH -- Calculate total revenue and growth revenue AS (SELECT...	Success	View
May 30 03:50 PM	fastfood_db postgres	Chat	Fix the execution error for my query.	-	View
May 30 03:48 PM	fastfood_db postgres	Execution	WITH -- Calculate total revenue and growth revenue AS (SELECT...	Success	View
May 30 03:48 PM	fastfood_db postgres	Chat	Yêu cầu: Báo cáo Đánh giá Hiệu suất Kinh doanh & Hành vi Khách...	-	View
May 30 03:48 PM	fastfood_db postgres	Execution	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	Success	View
May 30 03:48 PM	fastfood_db postgres	Chat	WITH store_revenue AS (SELECT o.store_id, SUM(o.total_after_d...	-	View
May 30 02:45 PM	fastfood_db postgres	Execution	WITH -- Calculate total revenue for Q4 2025 store_revenue AS (S...	Success	View

Picture 3.19: Global History and Activity Audit Log

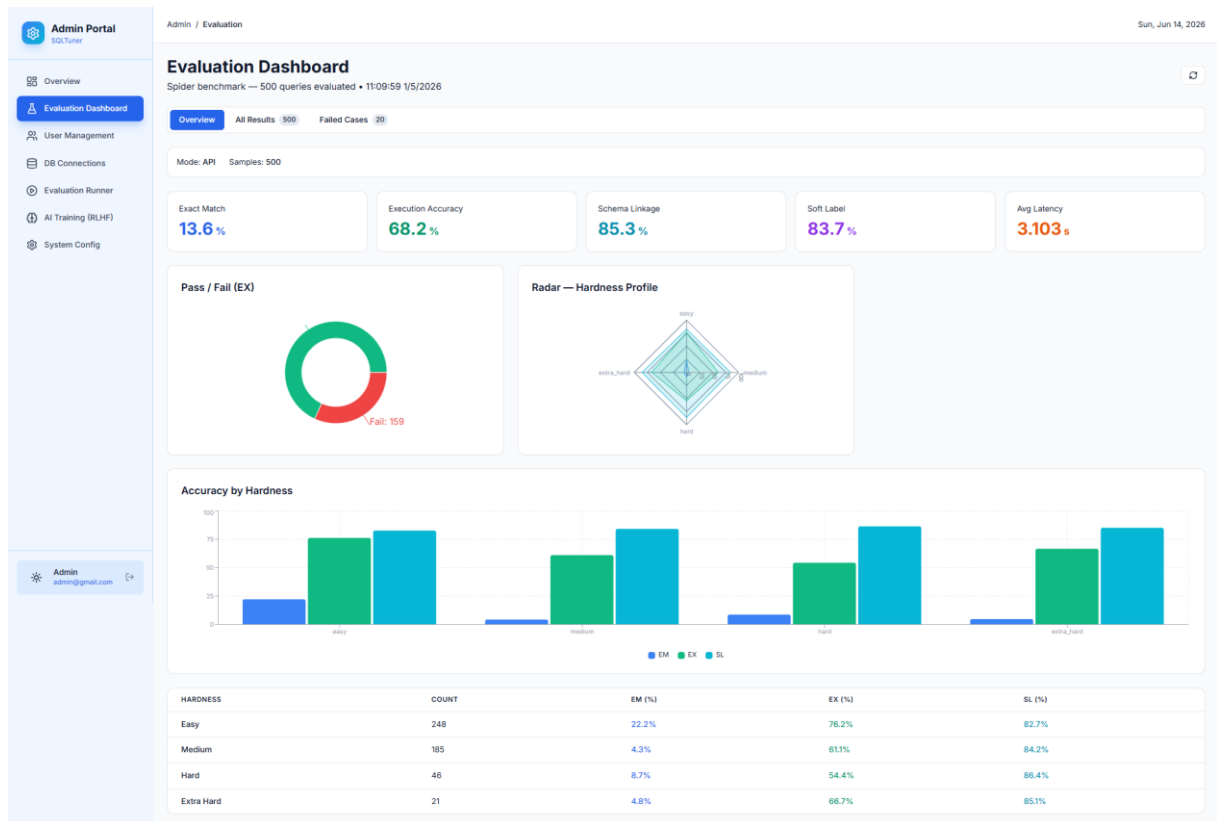
The Global History interface provides a comprehensive audit trail of all user activities across every workspace. This centralized log meticulously records the timestamp, targeted workspace, activity type (such as Chat prompt or SQL Execution), the specific query or natural language request, and the execution status. Users can easily search through past activities using keywords or filter the logs by specific workspaces and activity types, allowing for efficient tracking, auditing, and retrieval of previous AI interactions and executed scripts.

3.2.2. Admin Interface



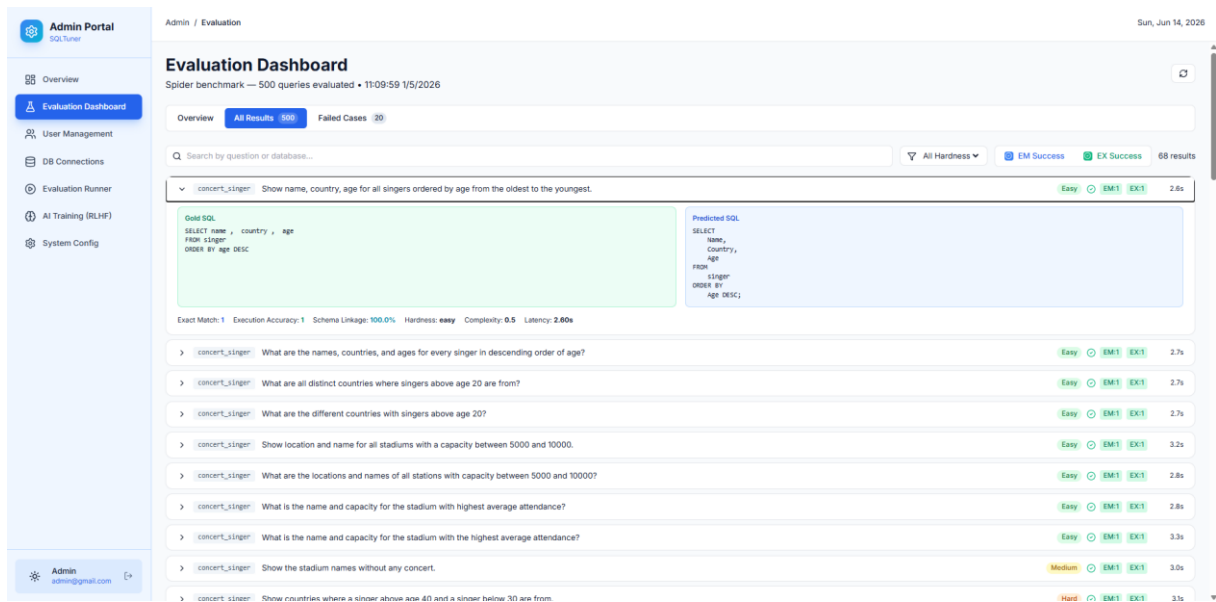
Picture 3.20: Admin Portal System Overview Dashboard

Transitioning to the Admin Interface, this image displays the primary System Overview dashboard for platform administrators. It provides a macro-level perspective of the SQLTuner system's health and operational metrics. Administrators can monitor key performance indicators (KPIs) in real-time, such as the total number of registered users, cumulative SQL query count, active database connections, and user feedback. The dashboard also features visual charts tracking query volume trends, alongside a comprehensive 'Recent Activity' log to audit system-wide events and user interactions.

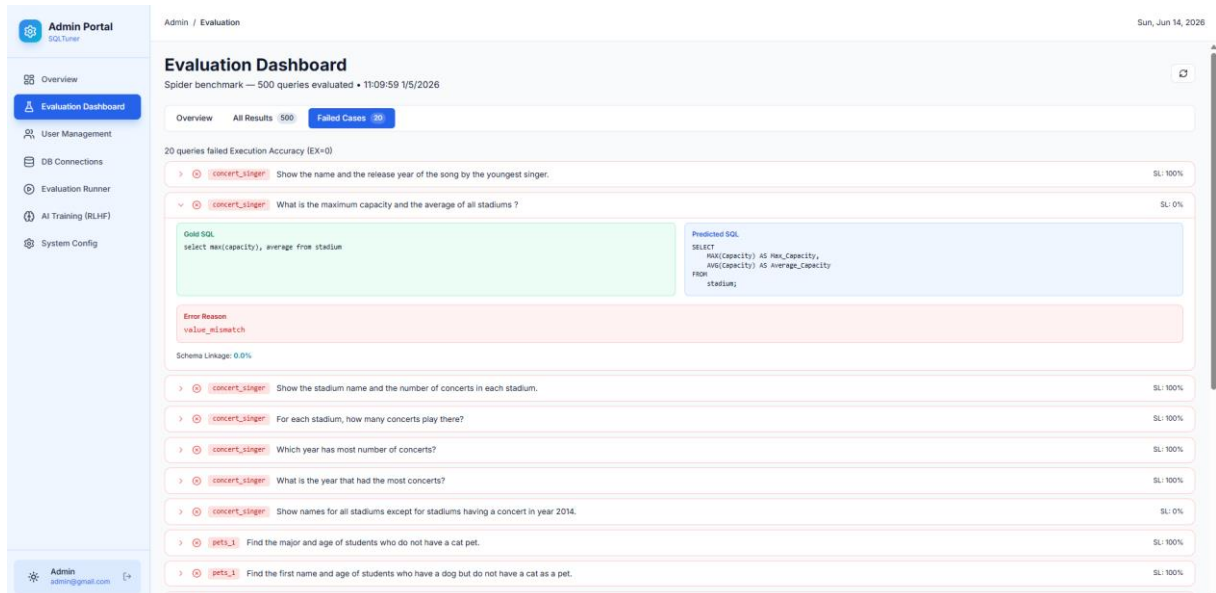


Picture 3.21: Admin Evaluation Dashboard and Benchmark Performance Metrics

This interface presents the Evaluation Dashboard within the Admin Portal, specifically designed to visualize the AI model's performance on standard academic benchmarks, such as the Spider dataset. It prominently displays core quantitative evaluation metrics, highlighting Execution Accuracy (EX) and Exact Match (EM) scores. Administrators and researchers can deeply analyze the model's proficiency through detailed visual charts, including a Pass/Fail distribution and a Hardness Profile radar chart that breaks down accuracy across different query difficulty levels (Easy, Medium, Hard, and Extra Hard). This comprehensive dashboard is essential for tracking model regressions and validating the effectiveness of new prompt engineering strategies.

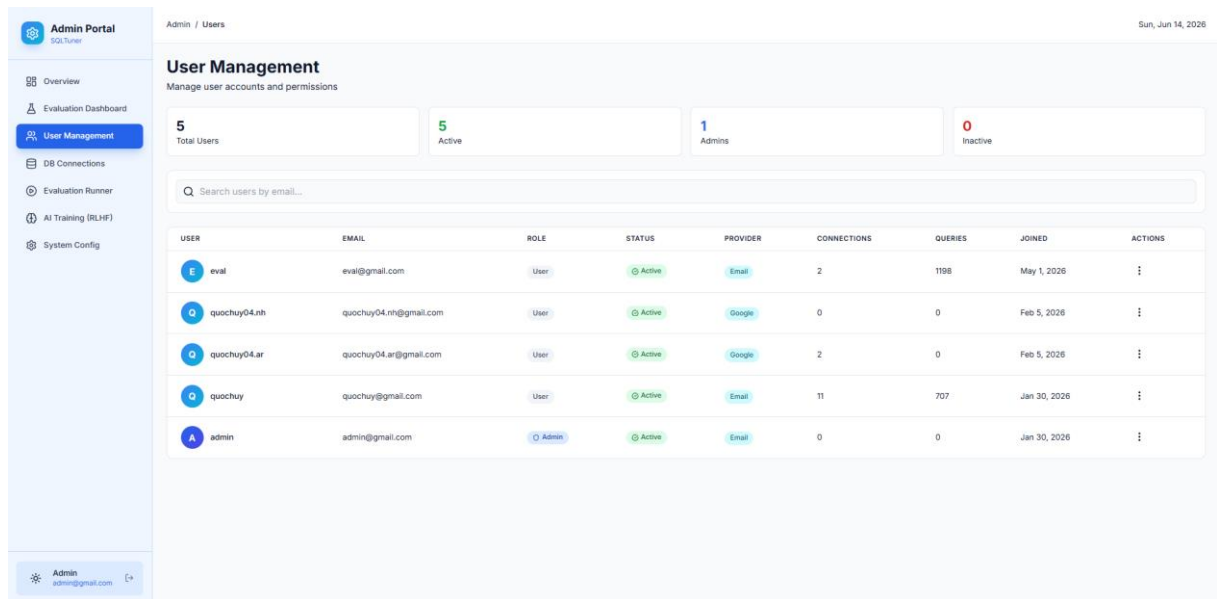


Picture 3.22: Success Evaluation Results and SQL Comparison on Spider Benchmark



Picture 3.23: Failed Evaluation Cases and Error Analysis on Spider Benchmark

This interface provides a granular view of the AI model's evaluation against the standardized Spider benchmark dataset. By navigating to the 'All Results' tab, administrators can drill down into individual test cases. The system presents a direct, side-by-side comparison between the expected 'Gold SQL' (the ground truth from the dataset) and the 'Predicted SQL' generated by the LLM. Each evaluated query is rigorously assessed and tagged with specific performance parameters, including Exact Match (EM) status, Execution Accuracy (EX) status, the query's hardness level, and execution latency. This detailed breakdown is vital for researchers to identify specific failure patterns and refine prompt engineering techniques.



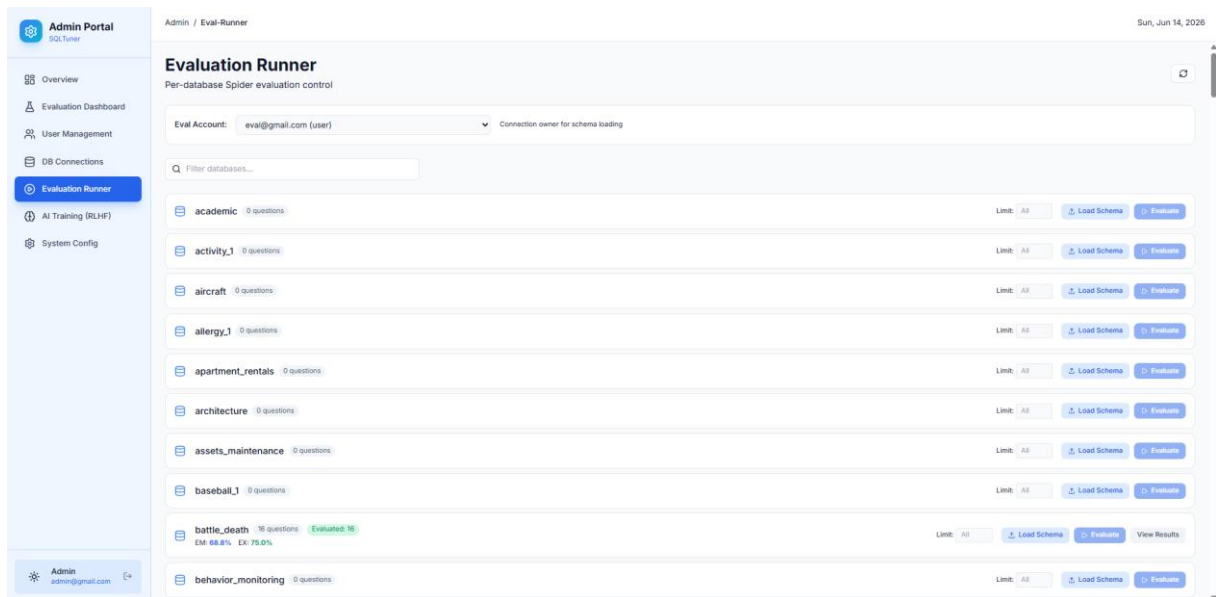
Picture 3.24: User Management and Role Assignment Control Panel

This interface represents the User Management control panel within the Admin Portal. It provides administrators with a centralized view of all registered accounts, displaying key details such as user roles, authentication providers (Email or Google OAuth), activity status, and engagement metrics like total connections and queries. Through this panel, administrators can easily search for specific users, audit their activity levels, and perform critical account management tasks, such as modifying access roles or deactivating accounts to ensure system security and proper access control.

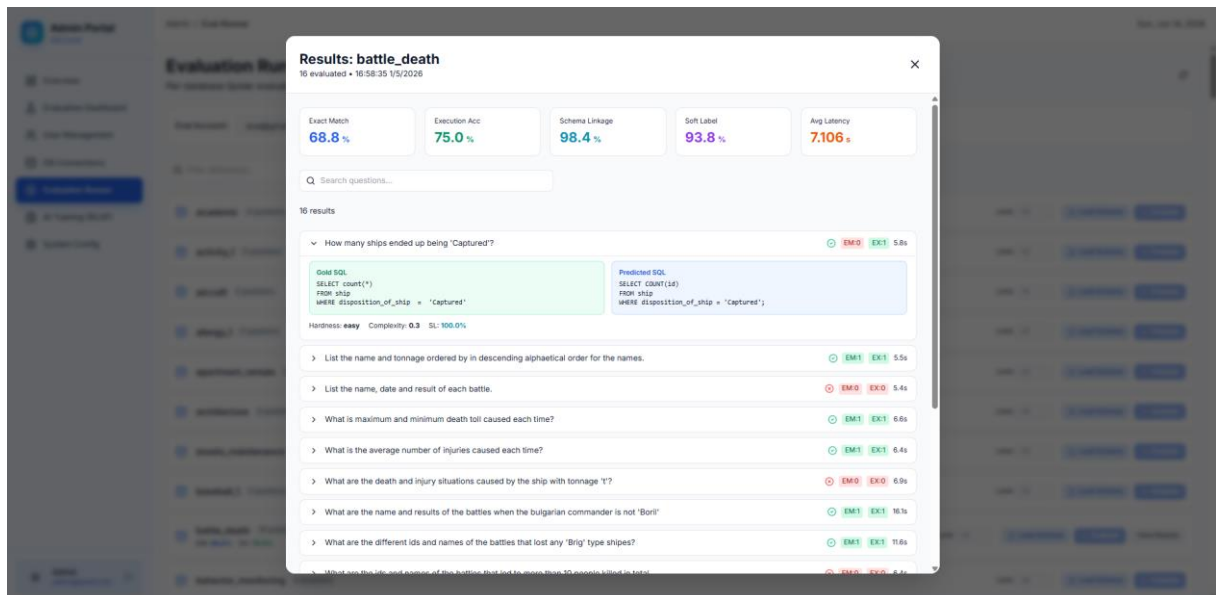
CONNECTION	TYPE	OWNER	HOST	DATABASE	TABLES	CREATED
spider_eval_battle_death	Simulation	eval@gmail.com	local	—	3	May 1, 2026
spider_eval_car_3	Simulation	eval@gmail.com	local	—	6	May 1, 2026
eval	Simulation	quochuy@gmail.com	local	—	4	Apr 30, 2026
evaluation	Simulation	quochuy@gmail.com	local	—	3	Apr 30, 2026
gen	Simulation	quochuy@gmail.com	local	—	5	Mar 24, 2026
ff	Postgres	quochuy04.ar@gmail.com	192.168.1.190-5432	fastfood_data	26	Feb 21, 2026
fastfood	Postgres	quochuy04.ar@gmail.com	192.168.1.190-5432	fastfood_data	26	Feb 21, 2026
simulation2	Simulation	quochuy@gmail.com	local	—	26	Feb 4, 2026

Picture 3.25: Global Database Connections Management Panel

This interface provides administrators with a comprehensive overview of all database connections and virtual schemas established across the platform. The dashboard displays aggregated metrics, categorizing connections by type, such as PostgreSQL, MySQL, and Simulation workspaces. The detailed data table allows administrators to audit individual connections, track ownership, review host details, and monitor the number of tables within each schema. This centralized visibility is crucial for maintaining platform security, monitoring resource usage, and ensuring proper data governance.

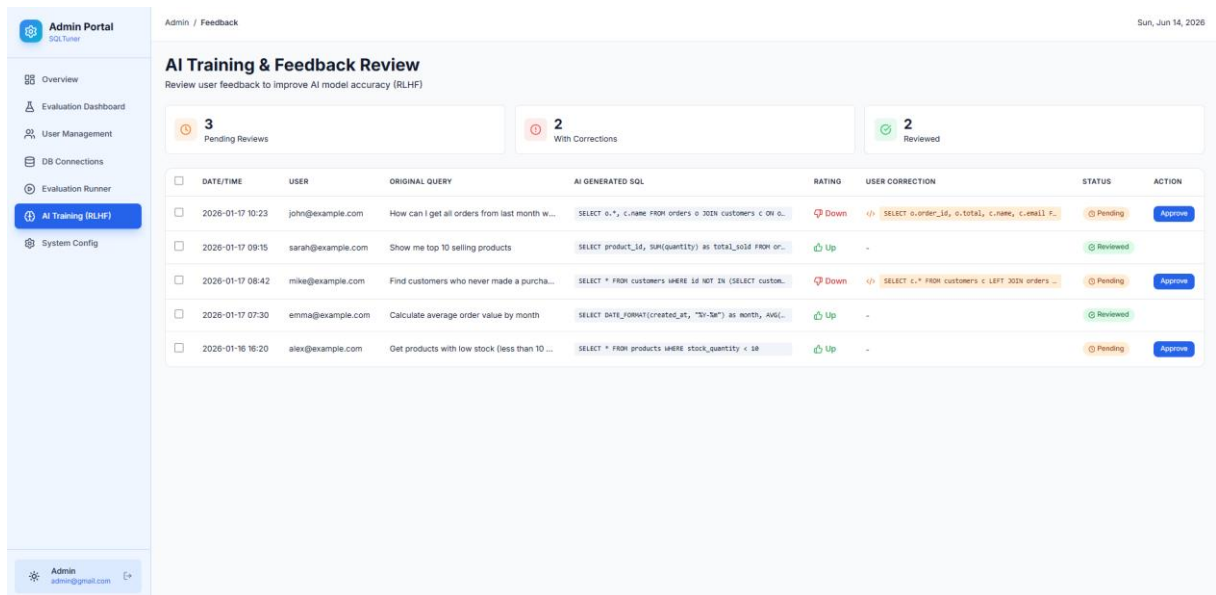


Picture 3.26: Evaluation Runner for Per-Database Benchmark Execution



Picture 3.27: Detailed Evaluation Results Modal for Specific Databases

The 'Evaluation Runner' interface empowers administrators to conduct highly targeted, per-database automated testing using the Spider benchmark dataset. As shown in PICTURE 3.25, users can select a specific evaluation account, load the corresponding database schema (e.g., 'battle_death'), and trigger the batch evaluation process. Upon completion, PICTURE 3.26 illustrates the detailed results modal. This modal breaks down the AI model's performance on that specific database, displaying aggregated metrics like Execution Accuracy (EX) and Exact Match (EM) scores, along with a granular, side-by-side comparison of the 'Gold SQL' versus the 'Predicted SQL' for every evaluated question.



Picture 3.28: Proposed AI Training and User Feedback Review Dashboard (RLHF)

This interface represents a planned future enhancement for the SQLTuner system: the AI Training and Feedback Review module, conceptually based on Reinforcement Learning from Human Feedback (RLHF). Although currently in the design and prototyping phase and not yet deployed for real-world testing, this dashboard envisions a continuous improvement workflow. It illustrates how administrators will be able to review user ratings (thumbs up/down) and manual SQL corrections submitted during chat sessions. By systematically collecting, validating, and approving these user-provided corrections, the system aims to construct a high-quality dataset to periodically fine-tune and retrain the underlying Large Language Model, thereby progressively enhancing its accuracy and context awareness over time.

3.3. Experimental Results

Experiments validated the system across 5 scenarios: Basic Text-to-SQL Generation, Complex Query Optimization, Batch Execution on Benchmarks (handling OOM gracefully), Evaluation Report Extraction, and Secure Execution with Database Simulation (catching destructive queries).

3.3.1. Scenario 1: Basic Text-to-SQL Query Generation

Objective: Test the responsiveness and syntax accuracy of the Generator module.

Result: The Local LLM successfully received Prompts and generated complex composite queries (SELECT, JOIN, GROUP BY). Response speed was good, and generated Database Schema fields perfectly matched actual provided structure without hallucination.

3.3.2. Scenario 2: Complex Query Optimization

Objective: Evaluate the Optimizer module's capacity to read Execution Plans and propose Database cost reduction solutions.

Result: Fed a poorly structured nested SQL Query causing a Sequential Scan, the system executed EXPLAIN (ANALYZE, FORMAT JSON). The LLM analyzed bottlenecks and proposed rewriting WHERE filters and adding DDL Indexes. Implementing these recommendations vastly decreased table scan time and Total Cost.

3.3.3. Scenario 3: Batch Inference on Benchmarks

Objective: Assess the Pipeline's load-bearing capacity and stability during high-volume Batch Evaluation runs on Spider and BIRD.

Result: Handled thousands of Natural Language queries via Queue Management mechanisms flawlessly, avoiding Out of Memory (OOM) and hardware overload through automated batching and memory release.

3.3.4. Scenario 4: Extracting Evaluation Reports

Objective: Visualize and analyze model result metrics.

Result: The Dashboard beautifully visualizes the Execution Accuracy (EX) and Exact Match (EM) metrics. Errors are distinctly categorized into Syntax Errors (LLM) vs Runtime Errors (PostgreSQL logic), robustly supporting developers in Debugging and Prompt Engineering refinement.

3.3.5. Scenario 5: Safe Execution with Database Simulation / Sandbox

Objective: Test fault tolerance and data protection in the Sandbox environment against destructive commands.

Result: AI-generated destructive SQL Queries (e.g., DROP TABLE) were immediately confined within a Transaction block, triggering the ROLLBACK mechanism. The origin Database remained unharmed, proving the system's safety in evaluating untested AI models.

3.4. Result Evaluation Remarks

Across all experimental scenarios, SQLTuner proved highly stable and exact. The Optimizer showcased substantial real-world value for Data Engineers and DBAs. Most importantly, the Evaluation Framework module operated precisely as designed, offering quantitative scientific metrics (EX, EM) to independently and securely evaluate Local LLMs against large, complex Database Schemas offline.

CONCLUSION

The "SQLTuner - AI Assistant for Automating Database Performance Optimization" project has successfully addressed the fragmentation in traditional code generation tools by tightly integrating intelligent query generation, performance optimization, and automated batch evaluation into a single platform.

1. Achieved Results

- **Accurate Text-to-SQL Generation:** By leveraging Local LLMs and rigorous prompt engineering (Schema Linking, Few-shot Prompting), the system effectively translates natural language into executable complex SQL queries without hallucinating schema structures.
- **Integrated Performance Optimization:** The Optimizer module successfully extracts metrics from Execution Plans via EXPLAIN ANALYZE, identifies bottlenecks like Sequential Scans, and automatically proposes actionable indexing and structural recommendations to minimize computational costs.
- **Robust Automated Evaluation Framework:** The system handles Batch Inference on standard academic benchmarks (Spider, BIRD) utilizing Queue Management and strict Batch Size control. This approach successfully maximizes hardware limits while perfectly preventing Out of Memory (OOM) scenarios.
- **Secure Execution Environment:** The Transaction-based Database Simulation Sandbox ensures complete isolation. It effectively catches destructive queries (e.g., DROP, TRUNCATE) and triggers immediate ROLLBACK mechanisms, preserving the zero-mutation state of the test database.

2. Core Contributions

SQLTuner delivers high practical value for two main user groups:

- **Backend Developers & DBAs:** Accelerating data retrieval tasks and providing deep insights into database bottlenecks.
- **AI Researchers & Data Engineers:** Providing a quantitative, scientifically backed framework using Execution Accuracy (EX) and Exact Match (EM) metrics to safely and accurately evaluate internal model lifecycles and prompt engineering strategies.

3. Limitations and Future Directions

- While the system efficiently optimizes queries for PostgreSQL, future updates should expand the Sandbox and execution plan analysis capabilities to cloud-based enterprise data warehouses, such as Google BigQuery.
- Investigate the application of Retrieval-Augmented Generation (RAG) to seamlessly integrate external knowledge bases. This will improve the

database-grounded understanding required to conquer highly complex, massive-scale queries in the BIRD benchmark.

REFERENCES

- [1] Jinyang Li, Binyuan Hui et al. (2023). *BIRD: A Big Bench for Large-Scale Database Grounded Text-to-SQLs*, *NeurIPS*. [arXiv:2305.03111](https://arxiv.org/abs/2305.03111)
- [2] Tao Yu, Rui Zhang et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*, *EMNLP*. [arXiv:1809.08887](https://arxiv.org/abs/1809.08887)
- [3] Sebastián Ramírez (2024). *FastAPI Documentation*, <https://fastapi.tiangolo.com/>
- [4] Gao, D., Wang, H., Li, Y., et al. (2023). *Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation*. *arXiv preprint*. <https://arxiv.org/pdf/2308.15363>
- [5] Pourreza, M., & Rafiei, D. (2023). *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction*. *arXiv preprint*. <https://arxiv.org/pdf/2304.11015>
- [6] Defog.ai (2024). *SQLCoder: State-of-the-art Large Language Models for SQL generation and analysis*. <https://defog.ai/blog/open-sourcing-sqlcoder>
- [7] DAC Data Technology Viet Nam (2026). *Internal Training Documentation: SQL Query and Optimization Techniques*.